IMPLEMENTING A POST-QUANTUM SECURE ADAPTOR SIGNATURE SCHEME IN BLOCKCHAINS

A dissertation submitted to The University of Manchester
for the degree of Master of Science
in the Faculty of Science and Engineering

2026

Royce Steven

Department of Computer Science

Contents

Word Count: 8963

List of Tables

List of Figures

Abstract

Blockchains authenticate transactions with elliptic-curve signatures that a large quantum computer would break. Basic post-quantum signatures are now standardised, but the *exotic* ones behind blockchains' advanced features, among them the *adaptor signatures* that make swaps and payment channels scriptless, remain unevenly implemented in the post-quantum setting. This dissertation implements one of them, LAS, by adding an adaptor layer and a wire encoding to the unmodified CRYSTALS-Dilithium reference primitives, cross-validated by a second, independent implementation reproducing the wire format and a pinned known-answer value byte for byte, and demonstrates it in a post-quantum atomic swap.

Against its own simplified Dilithium-style base at identical parameters and primitives, the adaptor layer is computationally cheap: PreSign, PreVerify and Adapt add +1.8%, +7.4% and +7.7% at the core boundary, rising to +19.7–87.8% once the wire codec is counted, an increase that is codec work rather than arithmetic. The real cost is communication: the signature is 99.3% response vector, an adaptor swap transmits a public-key-sized statement, and against a classical ECDSA adaptor the signature and public key are 72× and 89× larger. Rebuilding the adaptor path on ML-DSA as FIPS 204 specifies it roughly halves the signature but leaves the statement untouched, relocating that bottleneck rather than removing it.

At application scale the signature is not the dominant cost: across three configurations of a swap over a UTXO ledger, the zero-knowledge proof of knowledge takes 98.6–99.3% of end-to-end time in the two lattice configurations, only one of which is fully post-quantum. A restructured verifier also fits one Ethereum transaction, at the parameter set, message length and signature instance measured, and under an unchecked registration invariant. On this evidence the proof system, not the signature, is where such a swap should next be optimised.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Statement of qualifications and research

The author holds a Bachelor of Informatics from Institut Sains dan Teknologi Terpadu Surabaya, Indonesia, awarded in 2024. As part of this degree, the author completed an undergraduate final project on Indoor People Counter Using Wireless 802.11 Channel State Information.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Zhipeng Wang, for his guidance, support, and valuable feedback throughout this project. I am also deeply grateful to the Indonesia Endowment Fund for Education (LPDP) for its financial support and for providing me with the opportunity to pursue my Master's degree at the University of Manchester

Chapter 1

Introduction

1.1 Context and motivation

Public blockchains commonly authorise transactions with the Elliptic Curve Digital Signature Algorithm (ECDSA) or Schnorr. Their security relies on the hardness of the elliptic-curve discrete logarithm problem, which Shor's algorithm solves in polynomial time on a sufficiently large quantum computer [1], allowing an adversary to forge signatures and spend other users' funds. A published 2026 estimate puts secp256k1 key recovery within minutes at fewer than 500,000 physical qubits [2], and NIST's draft transition schedule would disallow quantum-vulnerable signatures at this strength after 2035 [3]. A chain cannot treat that as distant: its record is permanent, so a break reaches keys already exposed, and changing how it verifies signatures requires a coordinated consensus-rule change, not merely a local software update. "Post-quantum" (PQ) cryptography replaces this assumption with problems believed hard even for quantum computers, such as those over lattices.

The U.S. National Institute of Standards and Technology (NIST) has standardised *basic* PQ signatures, notably ML-DSA (the Module-Lattice-Based Digital Signature Algorithm), derived from CRYSTALS-Dilithium [4, 5]. A basic signature authenticates a message and nothing more. Blockchains, however, depend on *exotic* signatures that add functionality on top of a basic scheme: multi-signatures, aggregate signatures, threshold signatures, and *adaptor signatures* (fig. 1.1). A recent survey documents that their exotic counterparts remain unevenly served by practical implementations in blockchain settings [6]. Turning one such design, the LAS construction of Esgin et al. [7], into a tested, measured artefact is the problem this project addresses.

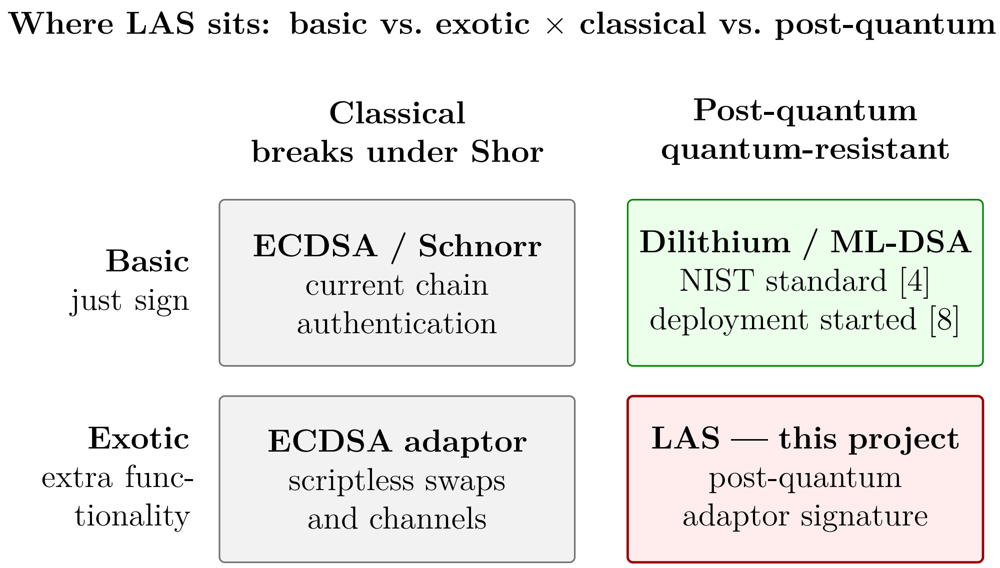

Figure 1.1: Where this project sits. Signatures divide along two axes: whether they survive Shor, and whether they do more than authenticate a message. Basic post-quantum signatures are standardised and deployment has begun, and the classical exotic cell is well served, but blockchain applications need exotic signatures that are also post-quantum. That remaining cell, holding the lattice adaptor signature this dissertation implements and measures, is the gap it addresses.

This dissertation focuses on the *adaptor signature*, which ties publishing a valid signature to simultaneously revealing a secret witness [9, 10]. That property makes adaptor signatures useful for scriptless blockchain protocols. In an atomic swap, two parties exchange assets across chains without a trusted intermediary (fig. 1.2). They also apply to payment-channel networks, linking conditional off-chain updates through the same witness mechanism. The condition is embedded in the signature, so settlement verifies as an ordinary signature and needs no on-chain script for that condition [7, 6]. The exotic family is large, so the choice needs a reason. Multi-signatures and threshold signatures change who must cooperate to authorise a spend. Aggregate signatures compress many signatures into one. An adaptor signature serves a different purpose: it links two *separate* settlements. Once an adapted signature is published, a party holding the matching pre-signature can extract the secret and use it to complete the other settlement. That is the property an atomic swap needs, and the other three primitives do not provide it by themselves. Common classical adaptor constructions rely on discrete-logarithm assumptions, so they do not survive Shor's algorithm. Practical post-quantum alternatives remain limited. This distinct function, the applications that depend on it, and the implementation gap are why the adaptor signature was selected.

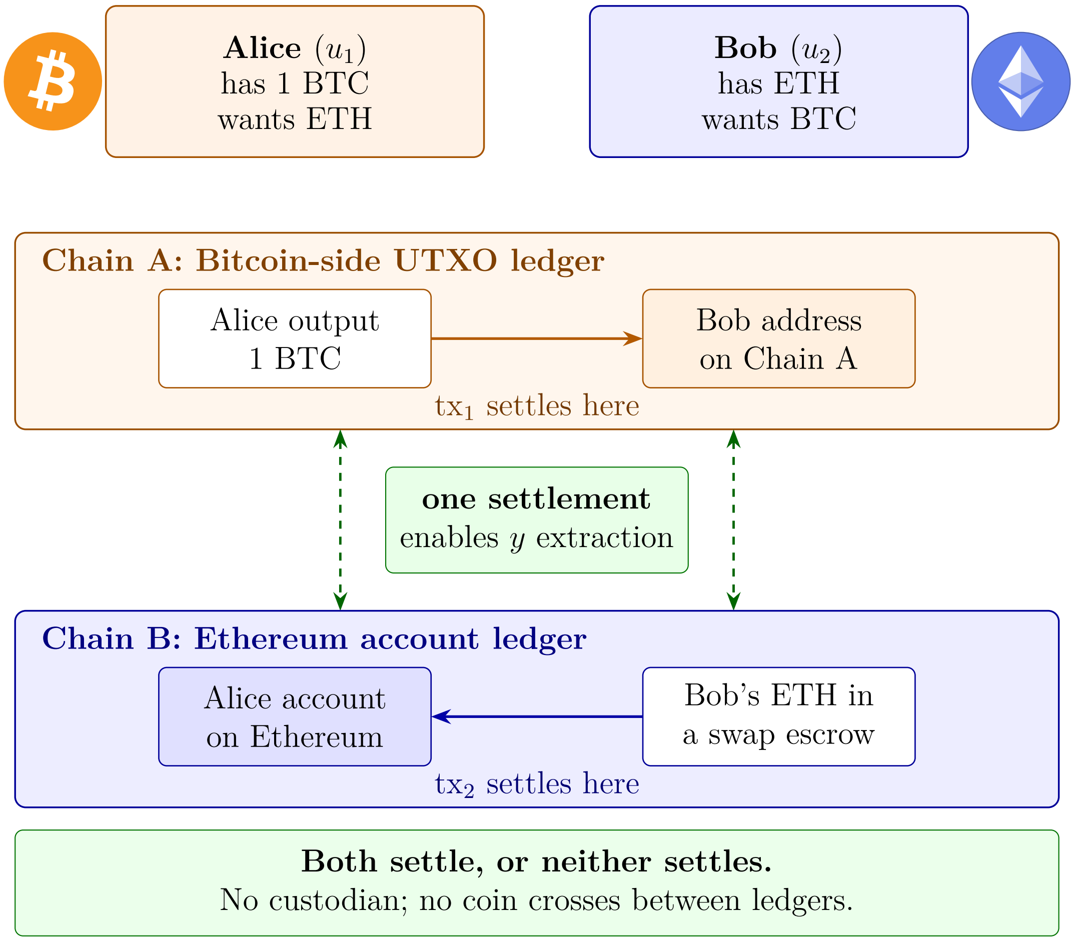

Figure 1.2: A motivation-level picture of an atomic swap, one concrete blockchain application of adaptor signatures. Alice and Bob trade assets recorded on two different ledgers: an unspent-transaction-output (UTXO) chain and an account-based one. Each payment settles on its own chain: tx_1 moves Alice's bitcoin to an address Bob owns, while tx_2 releases Bob's escrowed ether to an account Alice owns. The two venues place a lattice signature differently, which is why the Ethereum leg settles through a contract rather than as a plain payment (fig. 3.5). No coin crosses chains; the cross-chain object is the secret witness, which the counterparty extracts from the published signature using its matching pre-signature. Adaptor signatures provide this cryptographic link without requiring the swap condition to appear as a separate on-chain script. The two parties are named as in Esgin et al. [7], whose u_1 and u_2 the later chapters use; fig. 2.4 sets the exchange out message by message.

Cross-chain exchange first relied on custodial intermediaries; hash-time-locked contracts removed the custodian by locking both transfers under one hash preimage, so claiming one leg reveals the preimage unlocking the other. Such contracts need explicit on-chain script support, place a common hash on both chains (linking the legs) and enlarge the transactions. Moving the lock into the signature answers all three [9, 10]: settlements look like ordinary single-signer payments, atomicity is retained, and the witness reaches only the counterparty holding the matching pre-signature. The link removed is the explicit protocol one; timing and amounts can still correlate the legs.

1.2 Subject area and related work

Lattice signatures and Dilithium. CRYSTALS-Dilithium is a lattice signature in the "Fiat–Shamir with aborts" family [5, 11], its security reducing to the Module Learning With Errors and Module Short Integer Solution problems. It samples a masked commitment, derives a challenge by hashing, computes a response, and *rejects and retries* when the response would leak the secret: the rejection-sampling step characterising the family. Its reference implementation provides the polynomial arithmetic, number-theoretic transform (NTT) and SHAKE-based sampling this project reuses.

Adaptor signatures. Introduced informally as "scriptless scripts" [9] and later formalised [10], adaptor signatures add PreSign, PreVerify, Adapt and Extract to a base signature, relative to a hard relation (Y, y).

Post-quantum adaptor signatures. Esgin et al. [7] proposed LAS, a lattice-based adaptor signature on a Dilithium-style base; it is the design implemented here. Their work is primarily theoretical (definitions, a construction and security proofs), leaving the practical, benchmarked, blockchain-facing implementation open. A second post-quantum route is isogeny-based: Tairi et al. [12] build IAS on CSI-FiSh, prove it secure in the quantum random oracle model, and report an implementation with measured costs. Kysil et al. [13] put basic PQ signature verification on Ethereum, ruling out Dilithium for its key sizes.

Classical baseline. The natural answer to "what does post-quantum cost" is a comparison with a *classical adaptor* signature, not with plain ECDSA. Mature ECDSA-adaptor implementations exist [14]; this project benchmarks against one.

1.3 Aim and objectives

Aim. To implement and evaluate a post-quantum exotic signature scheme (a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives) and to demonstrate it in a post-quantum blockchain atomic-swap application. The work is positioned as *systems implementation and evaluation*: it does not propose a new cryptographic protocol.

Objectives.

1. Review post-quantum and exotic-signature literature sufficiently to justify selecting the adaptor signature LAS and the Dilithium primitives.
2. Implement LAS additively on the Dilithium reference, reusing the lattice core unchanged and adding the adaptor operations and a byte-level wire encoding.
3. Validate correctness and robustness through functional, serialization, tamper, and known-answer tests, including cross-validation by a second, independent implementation checked against the same known-answer value.
4. Benchmark LAS fairly (at explicitly stated parameters, with repeated runs and reported variance) against a simplified Dilithium baseline at identical parameters and a classical ECDSA-adaptor baseline, separating computation from communication cost across the five standard signature metrics.
5. Demonstrate an end-to-end post-quantum atomic swap using the implementation (following the protocol of [7], including its zero-knowledge proof of knowledge π) and discuss its feasibility.

1.4 Contributions

This dissertation contributes a tested, independently cross-validated implementation of a post-quantum adaptor signature, and measurements of its cost. Four findings carry it. At the core tier, adaptor functionality adds single-digit percentage overhead in computation over the simplified Dilithium-style base at identical parameters, but is expensive in communication (sections 3.2 and 3.4). The base in [7] simplifies Dilithium by omitting its hint and rounding machinery. On the full ML-DSA path, PreSign and PreVerify require adaptor-specific algorithms, yet an unmodified FIPS 204 verifier accepts the adapted signature (section 3.5). At application scale the zero-knowledge proof dominates execution time, leaving the fully post-quantum swap the faster of the two such configurations and the only one Shor does not break (section 3.7). Native on-chain verification first measured far above the transaction gas cap. Re-expressing the same predicate brings it under, at the one parameter set, message length and signature instance measured, and under an unchecked registration invariant. A settled swap sits well inside Bitcoin's standard-transaction weight limit (sections 3.8 and 3.9). All five objectives were met (table 4.1), subject to two limits throughout: no concrete security is formally analysed, and no experiment ran on a live network.

1.5 Dissertation structure

Chapter 2 describes the methods; chapter 3 reports the measurements and discusses their validity; chapter 4 judges the work against the objectives; chapter 5 concludes, reflects critically, and proposes future work.

Chapter 2

Methodology

2.1 From a base signature to a lattice adaptor signature

A signature scheme offers three operations: KeyGen produces a key pair, Sign signs a message, and Verify accepts or rejects. LAS keeps such a base signature and adds a public statement Y, its secret witness r', and four operations. A pre-signature is locked to Y: it passes PreVerify but not the base Verify until a party knowing r' unlocks it with Adapt, and publishing the unlocked signature lets anyone holding the pre-signature Extract the witness. Figure 2.1 shows that arrangement before the algebra.

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [7, 5]. Vectors and matrices are over $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, the public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n\times(n+\ell)}$, and a key pair is a short secret r with its image $t = A \cdot r$. The hard relation (Y, r') reuses exactly that structure, so a statement is structurally another public key: a sufficiently short preimage r' of Y under A yields a Module-SIS solution $(r', -1)^\top$ for $[\, A \mid Y \,]$, hence the relation's hardness [7]. Esgin et al. [7] name this statement-generation step Gen: sample an independent ternary vector r', compute $Y = A\, r'$, and return (Y, r'). Thus Gen is not a further primitive but KeyGen with its public key t relabelled as the statement Y and its secret r relabelled as the witness r'. *Key generation is shared.* The implementation mirrors that split: a relation module implements Gen, while the adaptor module in each language holds the four adaptor operations alone.

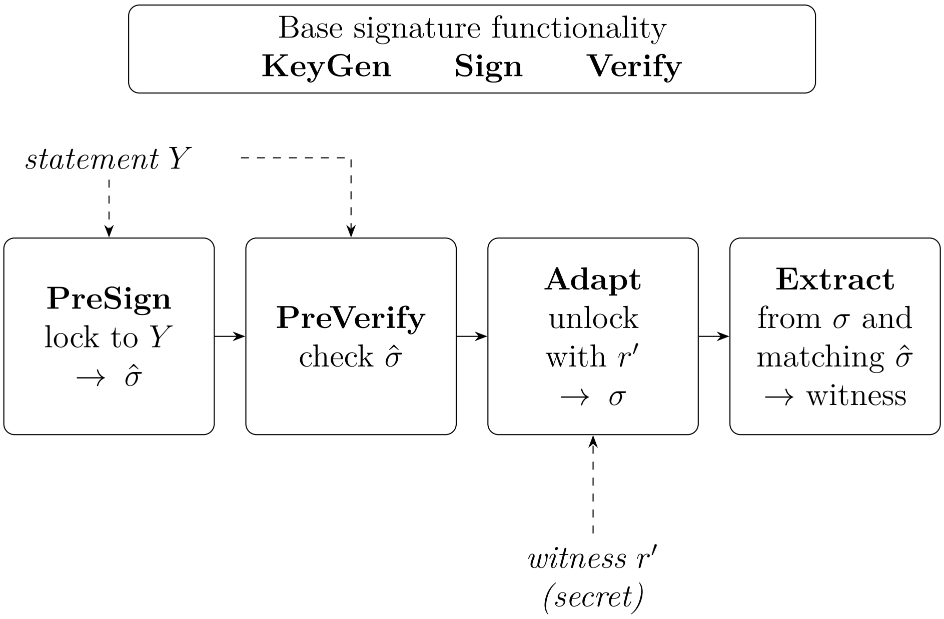

Figure 2.1: What changes when a base signature becomes an adaptor signature. Base Sign and Verify remain the signature interface; LAS adds PreSign and PreVerify, which lock signing to the public statement Y, Adapt, which unlocks a pre-signature $\hat{\sigma}$ using the secret witness r' to give an ordinary signature σ, and Extract, which recovers a witness from σ together with its matching pre-signature. Gen produces (Y, r') from the same key-generation relation. Extraction needing *both* objects is why publishing σ reveals the witness only to a party already holding $\hat{\sigma}$, which is the step an atomic swap uses. The algebra of each operation follows in the text.

The base signature commits to a mask y as $w = A\,y$, forms a challenge $c = H(pk, w, M)$, computes the response $\mathbf{z} = y + c\,r$ and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$. Relative to a statement Y, the adaptor extension [7] adds PreSign, which folds the statement into the hash, $c = H(pk, w + Y, M)$, keeps the response $\hat{\mathbf{z}} = y + c\,r$, and rejects at the *tighter* bound $\gamma - \kappa - 1$; PreVerify, which recomputes $w' = A\hat{\mathbf{z}} - c\,t$ and checks $c = H(pk, w' + Y, M)$; Adapt, which runs PreVerify and then adds the witness r', $\mathbf{z} \leftarrow \hat{\mathbf{z}} + r'$, producing a signature the base verifier accepts; and Extract, which recovers $s = \mathbf{z} - \hat{\mathbf{z}}$ and returns it only if $A\,s = Y$ (fig. 2.1).

Adding r' during Adapt imposes one constraint: the adapted response must still clear the base verifier's norm bound.

$$\text{norm condition: } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\,d\,(n + \ell). \tag{2.1}$$

An *honest* witness is ternary, so $\|r'\|_\infty \leq 1$, and because PreSign rejects at $\gamma - \kappa - 1$ the response after Adapt clears eq. (2.1); loosening it to $\gamma - \kappa$ could let adapted signatures exceed the bound and be rejected.

The construction therefore separates what an implementation may reuse from what it must add: the lattice arithmetic is common to both paths, while statement handling and the four adaptor operations form a layer above it.

2.2 Implementation strategy: reuse the lattice core, isolate the adaptor layer

That split determines the implementation strategy. The Dilithium polynomial, NTT and hash primitives are reused without modifying any upstream function, and the simplified base signature and the LAS operations are built around them. The implementation is built additively on the upstream CRYSTALS-Dilithium reference [5, 15] at a pinned commit, and the same strategy is applied again in Rust (section 2.5). Neither is itself "the reference": both are *modified, simplified* ML-DSA-style bases carrying the LAS layer of [7].

Figure 2.2 sets out that boundary, with the per-file classification in table A.7. The vendored `secp256k1-zkp` [14] and LaZer [16] are also used unmodified. LaZer is accessed through a dedicated bridge.

Data types and the matrix product. LAS is a self-contained module over the reference's degree-d polynomial type. Because $A = [\, I_n \mid A' \,]$, the product $A\, v = v_{\text{top}} + A' v_{\text{bot}}$ multiplies only A'.

Hash, challenge, and samplers. The random oracle H is built from SHAKE256, absorbing a canonical encoding of the public key, then the commitment (w for Sign, $w + Y$ for PreSign), then the message. The challenge sampler is the reference's `SampleInBall` at weight κ, giving $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary one for secrets and witnesses, and a rejection sampler uniform on $[-\gamma, \gamma]$ for the mask (appendix A.3).

Simplifications, and the modulus. Following [7], the base signature omits Dilithium's hint vector, Power2Round and high/low-bit decomposition, hashing the full commitment w rather than its high bits. This keeps the exact identity

$A\mathbf{z} - ct = w$ available, at the cost of larger keys and signatures. Section 3.5 tests whether those optimisations can coexist with an adaptor layer (design in appendix A.3). The modulus is reused too: the reference NTT fixes $q = 8\,380\,417 \approx 2^{23}$, FIPS 204's own modulus [4], rather than the $q \approx 2^{24}$ of [7]; correctness is unaffected since $q > 2\gamma$, and only the Module-SIS/LWE margin differs. No module depends on a mode-specific constant, so identical code runs at every parameter set.

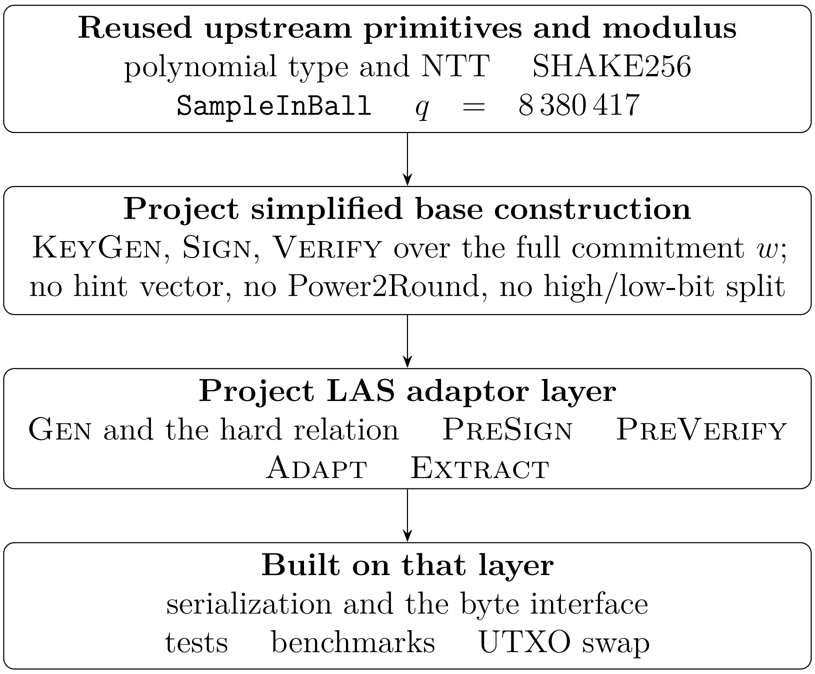

Figure 2.2: The implementation boundary, as a dependency stack. The lattice primitives are reused from the pinned upstream reference and *no upstream function is modified*. The simplified base signature is project code written over those primitives, not upstream Sign and Verify reused untouched: it omits the hint vector, Power2Round and the high/low-bit split so that the exact verification identity the LAS construction needs stays available. The adaptor layer is added above that base, and the serialization, testing, benchmarking and application work is built on the result. The per-file classification is table A.7.

2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs, so a bit-packed wire encoding, a typed codec and a verify-from-bytes entry point were implemented. Two decisions matter. Validation is *asymmetric*: keys and witnesses are checked on decode, a signature is not, so a tampered response is caught by Verify, the entry point the tamper test attacks. And, following FIPS 204 [4], a signature carries not the challenge polynomial c but the digest $\tilde{c}$ from which verification re-derives it, under the standard's $\lambda/4$ rule (appendix A.5). An on-chain verifier consumes that interface; section 3.9 evaluates one in Solidity.

2.4 Testing methodology

Four test types exercise different assumptions about the environment, stated in full in table A.1. Functional tests assume an honest caller and assert the whole adaptor cycle, including the *tripwire* clause (a pre-signature must *fail* basic verification), which is what distinguishes an adaptor signature from a signature. Serialization and tamper tests assume a hostile caller; the known-answer test assumes a different machine, and its pinned known-answer value is what the second implementation is checked against (section 2.5).

2.5 An independent second implementation: the Rust port

The complete scheme was implemented a second time in Rust on the same reuse principle, layered on a fixed version of the `fips204` crate [17], with the LAS layer added as new modules. The wire format is byte-identical to the C encoder's by construction (fig. 2.3).

The port serves two purposes. *Cross-validation*: short of a formal proof, a second implementation reproducing the pinned vectors is practical evidence of correctness. *An independent benchmark methodology*: Criterion.rs [18] shares no timing code with the C harness, so agreement provides evidence that the C results are not timing-harness artefacts.

To make the timings comparable the two protocol drivers mirror each other exactly: same repetition scheme, fixed seed, fixed message, success-path gating and rejection statistics. One caveat qualifies the port's independence: Rust follows the C implementation's early-abort norm check, an implementation detail not prescribed by the LAS construction [7]. Using the same early-abort check keeps the rejected-attempt work profile comparable across the two languages.

This was established by inspecting the two norm checks, not by the automated evidence, which constrains outputs and rejection rates but not rejected-attempt work (appendix A.3).

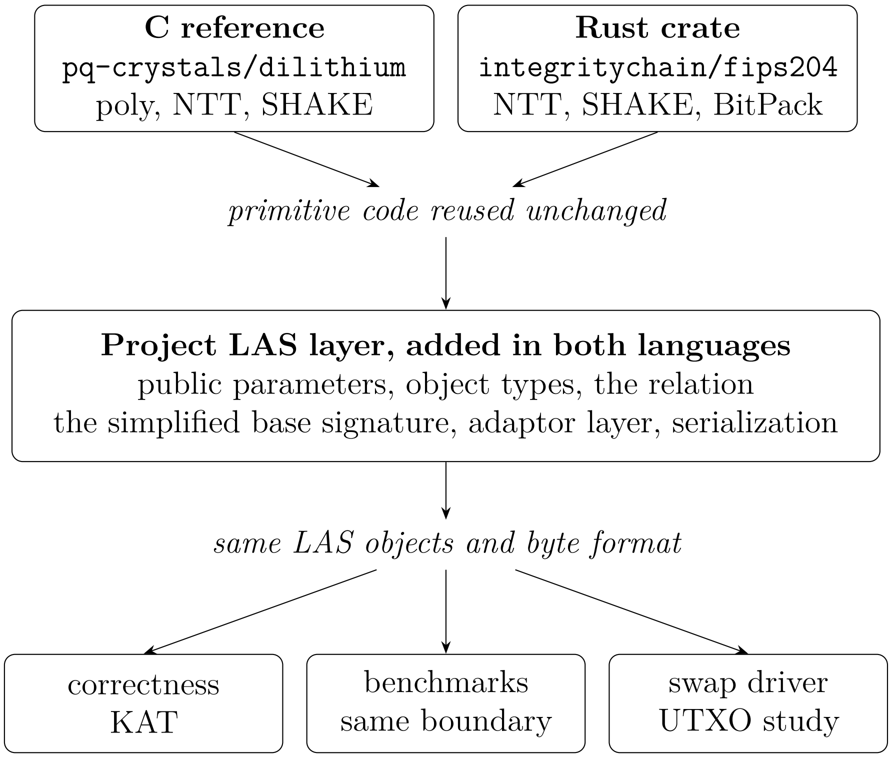

Figure 2.3: Implementation structure and C/Rust correspondence across the two implementations. C uses the vendored `pq-crystals/dilithium` reference [15], Rust uses the `integritychain/fips204` crate [17], and neither modifies upstream primitive functions. The project LAS layer is added in both languages with the same module split and packed object format; tests, known-answer vectors, matched benchmarks, and the UTXO swap driver are produced from that layer. The per-file classification is table A.7.

2.6 Benchmarking methodology and fair comparison

Measurement boundaries. The evaluation separates *computation* cost (wall-clock time per operation) from *communication* cost (bytes transmitted or stored). Undocumented boundaries are a recognised benchmarking pitfall for lattice signatures [19], so every timing belongs to one of the three declared boundaries of table A.3: a *core tier* isolating the adaptor algebra from encoding, a *packed tier* adding the wire codec a consumer reading from bytes must run, and, for the classical baseline alone, the *hybrid* boundary its library exposes. Comparisons stay within one boundary; the primary base-versus-LAS comparison shows *both*.

Implementation of record, repeatability, and machine. Unless a caption says otherwise, timings are the C implementation's, with the Rust results reported separately and never averaged with them (table 3.3). Driver timings are a mean $\pm$ sample standard deviation over each caption's repetitions; Criterion reports bootstrap 95% confidence intervals. All non-random inputs are fixed, so variability comes only from rejection sampling and machine noise, never from input selection [19]. Toolchain, hardware and per-table commands: appendix A.1.

Per-operation reporting, not cumulative. Operations are timed separately because the parties pay them separately; a cumulative figure would hide the per-operation overhead.

The primary comparison: LAS against its own simplified base. LAS is exactly its base signature plus four operations at identical algorithm, parameters and primitives, so each base-versus-LAS comparison holds all three fixed and varies only the adaptor layer (justified in section 4.1). Each adaptor operation is paired with the base operation it mirrors (Adapt against Verify, since it runs a PreVerify before adding the witness), and Extract, having no base analogue, is reported separately.

Rejection sampling: the expected attempt count. Sign-class operations restart until the response passes its norm bound, so no timing claim is meaningful without the attempt statistics. Sign accepts each coefficient with probability $\big(2(\gamma-\kappa)+1\big)/(2\gamma+1)$ *independently of the secret* — why the response is simulatable (appendix A.3) — and an attempt succeeds only if all $(n+\ell)\,d$ pass:

$$p_{\mathrm{acc}} = \left(\frac{2(\gamma-\kappa)+1}{2\gamma+1}\right)^{(n+\ell)d} \approx \left(1-\frac{\kappa}{\gamma}\right)^{(n+\ell)d} \approx e^{-\kappa(n+\ell)d/\gamma} = e^{-1} \approx 36.8\%, \tag{2.2}$$

the last step using the design choice $\gamma = \kappa\, d\,(n+\ell)$ from [7], made exactly so the expected number of restarts is "about $e < 3$". Expected attempts are $1/p_{\mathrm{acc}}$, and

PreSign's tighter bound gives it a slightly higher expected attempt count, which must not be read as adaptor arithmetic.

Rejection sampling: the run-validity gate. Because the attempt count is geometric, per-call timings are right-skewed and averages over too few calls can drift several percent on rejection luck alone [19]. The drivers therefore treat the attempt statistics as a *validity gate*: measured mean attempts must match the closed-form expectation within five standard errors, and a failing run aborts instead of producing evidence. This is a consistency check on the rejection rate, not a proof of sampler correctness (appendices A.2 and A.3). A second gate precedes every timing: the correctness contract is asserted on the objects about to be timed, so no failure path is ever measured.

Parameter sensitivity and component sizes. To test the adaptor overhead's sensitivity to the parameter set, the same comparison is repeated at the four parameter sets of table 2.1 (symbols in table A.2). Each key and signature is decomposed into components, so the *cause* of a size difference can be identified, not merely its total stated.

The classical baseline: functionality-matched, not security-matched. The second baseline is the `secp256k1-zkp` library's `ecdsa_adaptor` module [14], implementing Fournier's construction [20], unmodified at a pinned commit. It is a functionality-matched "price of post-quantum" baseline, not a same-security one; plain ECDSA is excluded, the fair counterpart of an adaptor signature being another adaptor signature.

2.7 Application methodology: the UTXO swap study

The venue, and why it is modelled. The construction [7] states its applications over "a UTXO-based blockchain like Bitcoin *where the signature algorithm is replaced with a lattice-based signature scheme*". That sentence is implemented literally: a UTXO ledger was built in which *the signature algorithm is a parameter*, so one ledger serves every configuration and any measured difference is the

Table 2.1: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set takes its dimensions from Esgin et al. [7]. It is not an exact reproduction of that parameter set: it keeps those dimensions as a historical reference point, corresponding to no ML-DSA set, and runs at FIPS 204's $q \approx 2^{23}$ [4], while Esgin et al. [7] set $q \approx 2^{24}$. Correctness is unaffected ($q > 2\gamma$), while the security margin changes. The *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5, aligning the reusable ML-DSA primitives and the challenge-digest strength $|\tilde{c}|$ with ML-DSA-44/65/87 while retaining LAS's own ternary secret distribution, rejection bounds, exact hint-free relation and unoptimised serialization. They are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims: a matched dimension is not a claim that LAS *is* the corresponding ML-DSA parameter set or inherits its security category. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table A.2 defines each symbol.

Setting	n	ℓ	M	κ	γ	d	q
LAS-2020/845 reference	4	4	8	60	122 880	256	8 380 417
Simplified Dilithium-II	4	4	8	39	79 872	256	8 380 417
Simplified Dilithium-III (target)	6	5	11	49	137 984	256	8 380 417
Simplified Dilithium-V	8	7	15	60	230 400	256	8 380 417
Classical (secp256k1 ECDSA adaptor)	≈128-bit classical (see table 3.5)						

cryptography, not the ledger. It is a model, not a node; the boundary is set out in appendix A.4.

The protocol that is executed. Figure 2.4 is the run book implemented by the driver. Alice, u_1 in [7], holds the witness y and therefore moves first; Bob, u_2, only pre-signs after Alice's pre-signature and, in the LAS configurations, her proof π have been verified. That proof is a required step of the method: it proves the statement has a short witness, closing the knowledge gap that would otherwise let a malicious first party leave an unadaptable pre-signature behind. No UTXO is spent while the pre-signatures are exchanged: a settlement occurs only when a transaction carrying an adapted signature is broadcast to its own ledger. The other ledger never verifies a foreign transaction; atomicity comes from Bob observing σ_2 on ledger B, extracting the witness, and using it to finish the spend on ledger A. The signed message m_i is the template tx_i serialized without its input signatures, not the settled transaction bytes.

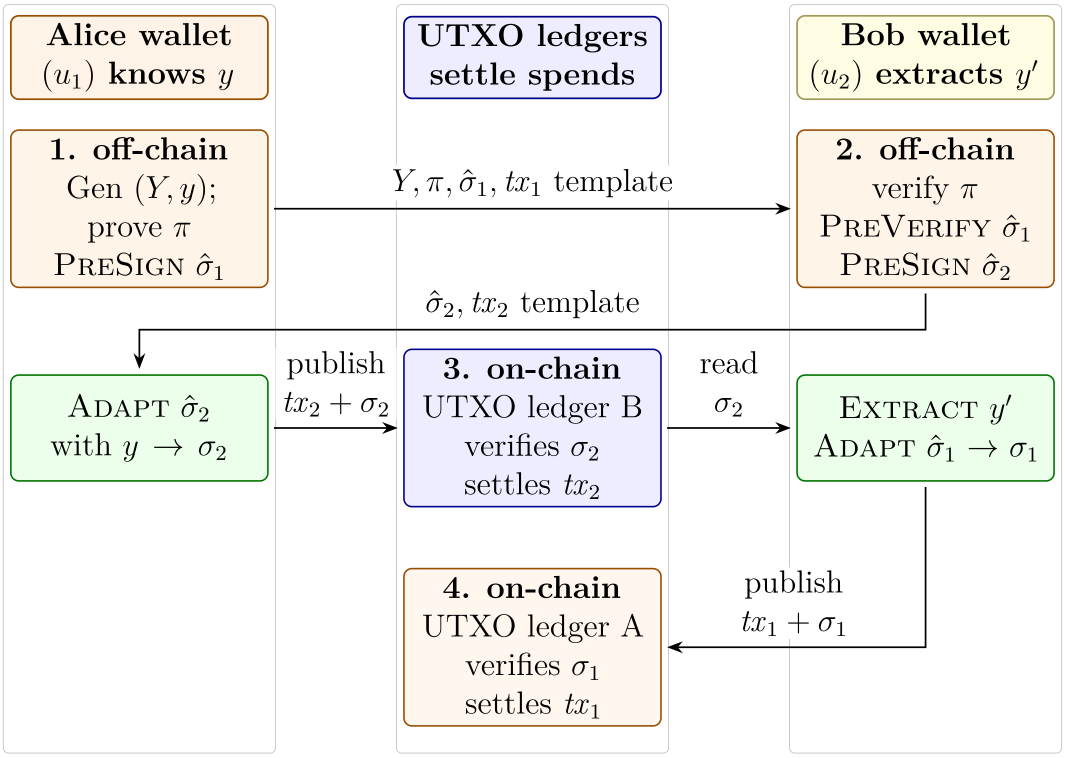

Figure 2.4: The atomic-swap protocol used in the UTXO study, adapted from Figure 1 in [7]. Protocol messages are exchanged off-chain; only the completed spend transactions are submitted to the two UTXO ledgers. The secret witness is never transmitted.

What a settled transaction contains, and what the adaptor layer changes. Of the adaptor-protocol objects exchanged by the swap, only the adapted signature σ reaches a ledger, occupying the slot in which an ordinary signature already appears: since SegWit [21], a segregated *witness* field. The statement Y, the proof π and both pre-signatures stay off-chain, while the witness y is never transmitted: u_2 derives it from the published σ and its own pre-signature. The construction therefore adds *no adaptor-specific on-chain field* — what *scriptless* means: the swap condition need not appear as a separate on-chain script. The resulting transaction cost is not modelled but read from two *mined* regtest transactions: the client's own size, weight and virtual size, with the field decomposition reconstructed from the decoded transactions and checked against the weight accounting of [21, 22, 23] (section 3.8). Those two transactions come from different stages: carriage past a stock node, and settlement under an experimental consensus rule on a patched one (appendix A.11).

One notational point matters once the venue is a real ledger. The swap protocol writes the signed object as tx_i, but a transaction is not a message: a signature commits to its *signature hash*. Three objects are therefore kept distinct: the unsigned template exchanged off-chain, the message derived from it that PreSign receives, and the settled transaction. Conflating them would misreport communication cost.

Table 2.2: The three configurations. Only the $2 \to 3$ step is controlled, so those two differ in the proof system alone; configuration 1 differs in signature scheme, relation and proof alike. Its missing role-A proof is a finding, not an omission: an ECDSA adaptor has no knowledge gap, since extraction recovers the discrete logarithm exactly, so the unmodified `secp256k1-zkp` construction needs no π, and adding one would have measured a modified baseline, not the library as it stands. The DLEQ proof carried *inside* its pre-signature is a different object and is not counted as π: its author is the pre-signer, and its claim is pre-signature well-formedness, not knowledge of the role-A statement witness r'. Its cost is included in the bytes by which a pre-signature exceeds a signature, which are not split between proof and structure: that excess carries the nonce commitments as well.

#	Signature	Role-A proof π	Setup	Fully PQ
1	ECDSA adaptor signature (libsecp256k1-zkp)	none required	—	no
2	LAS (simplified Dilithium-III)	Groth16 over BN254 (pairing-based zk-SNARK)	trusted	no
3	LAS (simplified Dilithium-III)	LaZer (LNP22 linear relation with norms)	transparent	yes

Three configurations, and what is controlled. The three configurations of table 2.2 each run the swap protocol end-to-end across two ledgers. The *role-A* proof is the proof of knowledge π the protocol places on the wire immediately after Y, claiming a witness to Y exists and is short. Only the $2 \to 3$ step is controlled: both LAS configurations share one expansion of the public matrix, derive all key material from one pinned seed, and prove the *identical* relation $\exists\, r' : Ar' = t' \wedge \|r'\|_\infty \leq 1$: the hard relation of the statement/witness pair, *not* the signer's key pair, and including the norm bound, since proving only $Ar' = t'$

would fail to close the knowledge gap. Each half computes the *relation binding* independently, and the run aborts on disagreement (appendix A.3). The $1 \to 2$ step is not controlled (it changes the signature scheme, the relation and whether a role-A proof exists), so it is a whole-stack comparison, the signature-only question being answered by section 3.4.

The Groth16 circuit. No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and cheaper than the statement suggests: $r' \mapsto Ar'$ is *linear with public coefficients* (appendix A.3).

Measurement. Because gas does not exist on this venue, the axes are execution time and communication cost, the latter counting *off-chain* protocol messages, not only what reaches a ledger. Sizes are observed from the transmitted buffers rather than computed, and these timings sit at the packed tier, so they are not comparable with core-tier figures elsewhere. One-time costs are excluded and reported separately (appendix A.3).

Chapter 3

Results and discussion

3.1 Correctness and robustness

No performance claim means anything until the implementation is shown *correct*, and an adaptor signature is not correct merely because it signs and verifies; it must satisfy a specific contract. Table 3.1 lists every clause; all pass. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire), becomes an ordinary signature once the witness is added, and completing it reveals that witness: the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness, clause 7 reproducibility.

3.2 Computation cost

The primary computation result is the cost of the *adaptor layer*. With algorithm, parameters and primitives fixed, the paired comparison estimates that cost (section 2.6). Table 3.2 reports it at *both* boundaries and fig. 3.1 visualises the core tier, where every paired adaptor operation at the target setting stays within single digits of its counterpart, Extract is the cheapest of all, and every operation completes in well under two milliseconds.

At the core boundary the adaptor layer adds little work, and the reasons are structural. PreSign (+1.8% over Sign) runs the same reject loop, adding only Y to the commitment and a tighter bound; PreVerify (+7.4%) recomputes the same commitment shape with the extra $+Y$; Adapt (+7.7%) runs a PreVerify then adds the witness vector; Extract merely subtracts $s = z - \hat{z}$ and checks $A\,s = Y$. None changes the asymptotic work. *At the core tier* adaptor functionality is single-digit

Table 3.1: The adaptor-signature correctness contract at the target setting, with the test that checks each clause in the evidence run of record; all pass. Clauses 1–4 and 5a are asserted on every one of the 1000 randomised iterations of the functional suite; 5b and 6 by the serialization suite, which also performs 256 randomised serialization round trips; 7 by the known-answer test, whose pinned cross-language value binds the packed public key, secret key, signature, pre-signature and adapted signature over four fixed vectors. PreVerify and Extract are asserted in all three suites, but produce no object that value covers.

#	Property	Result
1	PreSign produces a pre-signature that PreVerify accepts	pass
2	the pre-signature *fails* basic Verify (statement-binding tripwire)	pass
3	Adapt yields a signature that basic Verify accepts	pass
4	Extract recovers the witness *exactly* ($s = r'$)	pass
5a	a tampered message fails Verify	pass
5b	a low-bit flip in each of the 6736 packed-signature bytes is rejected	pass
6	malformed bytes rejected on decode ($pk \geq Q$, ternary code-3), out-of-range objects on encode (z, non-ternary secret)	pass
7	the deterministic API is byte-identical on re-run	pass

at the target setting and stays within +7.7% across the parameter sweep. The supporting sensitivity plot is deferred to Appendix A.8.1, where fig. A.1 records the same comparison across all four parameter sets. Adapt and Extract cost about a verification or less each (table 3.2).

At the packed boundary the premium grows to +19.7%, +42.3% and +87.8%, several times the core-tier percentages. The increase is codec work on the adaptor's own objects, not adaptor arithmetic: PreSign and PreVerify additionally decode the statement, while Adapt also decodes the witness and re-encodes the signature. A deployment working from bytes should budget for the packed row; reported lattice timings are notoriously sensitive to this boundary [19].

The longer Sign and PreSign timings are amplified by the rejection sampling intrinsic to Fiat–Shamir-with-aborts. Equation (2.2) predicts 2.719 attempts per Sign and 2.775 per PreSign, the latter expected to restart about 2% more often because its bound is tighter by one; counted directly rather than estimated from a timing ratio, the run of record observed 2.662 and 2.817. Table A.4 and fig. A.2 compare the measured rejection behaviour with the geometric model. All measured runs fall within five standard errors of the expected attempt count defined in section 2.6. At 5000 timed calls the gate band ($\pm\sim$6%) is wider than

Table 3.2: The primary computation result: per-operation adaptor overhead at the target setting *Simplified Dilithium-III* (table 2.1), at *both* measurement boundaries of section 2.6. Each LAS adaptor operation is paired with the basic operation it mirrors; "Overhead" is the extra cost as a percentage of that basic operation, within the same tier. Timings are μs per operation, measured on the C implementation, the implementation of record (mean ± sample standard deviation over 5 repetitions × 1000/1000 iterations; machine of record in section 2.6); the Rust port's independent measurements are in table 3.3. KeyGen, Sign, and Verify are reused unchanged by LAS, and Gen executes exactly the same computation as KeyGen, so the measured KeyGen timing also gives the cost of relation generation. Extract has no basic analogue. The packed-tier overheads are larger because the byte API does codec work on the adaptor's own objects that the basic operation does not: codec work, not adaptor arithmetic, itemised per operation in the text; the core tier isolates the adaptor computation itself. Absolute core-tier timings at every parameter set are in table A.5 (appendix).

Operation (basic / adaptor)	Basic (μs)	LAS adaptor (μs)	Overhead
Core tier (structures in/out — isolates the adaptor arithmetic)			
KeyGen	45.9 ± 1.9	45.9 ± 1.9 (shared)	—
Sign / PreSign	417 ± 11	424 ± 6	+1.8%
Verify / PreVerify	95.3 ± 0.5	102 ± 2	+7.4%
Verify / Adapt	95.3 ± 0.5	103 ± 1	+7.7%
Extract	—	37.2 ± 0.1	(LAS only)
Packed tier (wire bytes in/out — includes decode and encode)			
KeyGen	154 ± 1	154 ± 1 (shared)	—
Sign / PreSign	660 ± 18	791 ± 7	+19.7%
Verify / PreVerify	327 ± 3	465 ± 2	+42.3%
Verify / Adapt	327 ± 3	614 ± 3	+87.8%
Extract	—	467 ± 1	(LAS only)

that 2% separation, so the core-tier PreSign overhead (+1.8%) should be read as of the same order as the prediction, not as a precise constant; only the $\approx 10^5$-call Criterion harness resolves it.

3.3 Cross-implementation check: C implementation versus Rust port

The scheme exists twice (section 2.5), and at the byte level the two implementations agree exactly: the same packed sizes (public key 4416 B, signature 6736 B, checked programmatically on every Rust run) and the same pinned known-answer

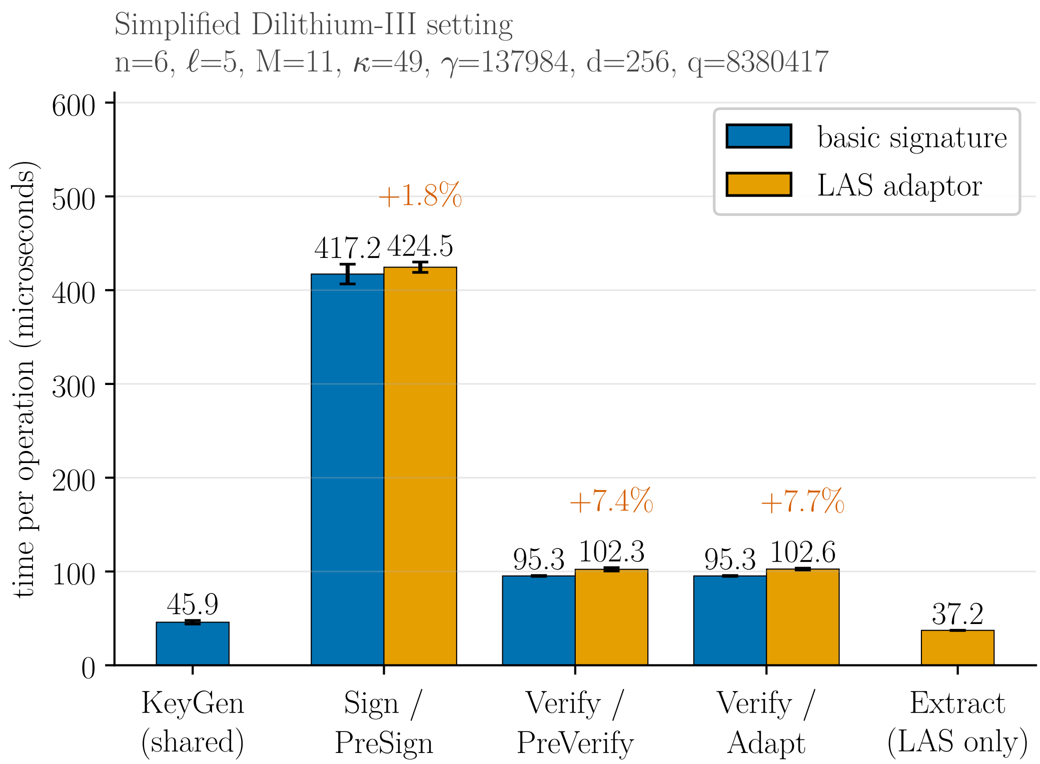

Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot), measured on the C implementation (the implementation of record, section 2.6). The percentage above each orange bar is the adaptor overhead over its blue partner. KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. Error bars are the sample standard deviation. Exact means, the repetition counts and the packed tier are in table 3.2; the same comparison across all four parameter sets is the parameter-sensitivity check in fig. A.1.

value. Since that value binds the packed key pair, signature, pre-signature and adapted signature, a divergence escapes it only by leaving every one of those bytes unchanged.

Absolute timings do not agree: neither port is uniformly faster, the largest gap is 14% (KeyGen), and no cause for it has been measured. What reproduces

Table 3.3: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* (μs per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions $\times$ 1000/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

Operation	C implementation (μs)	Rust port (μs)	Rust / C	Rust, Criterion (μs, 95% CI)
KeyGen	45.9 ± 1.9	52.3 ± 2.4	1.14	40.4 [40.4, 40.5]
Sign	417 ± 11	451 ± 12	1.08	369 [368, 371]
Verify	95.3 ± 0.5	95.8 ± 1.8	1.01	74.2 [74.2, 74.2]
PreSign	424 ± 6	457 ± 32	1.08	378 [376, 379]
PreVerify	102 ± 2	94.1 ± 1.3	0.92	74.6 [74.6, 74.7]
Adapt	103 ± 1	95.9 ± 1.3	0.93	76.0 [75.9, 76.1]
Extract	37.2 ± 0.1	37.9 ± 0.6	1.02	27.7 [27.7, 27.8]

is the comparison itself. At the core tier the Rust port measures +1.2%, −1.8% and +0.2%, all single-digit, like the C column beside them; the slightly *negative* PreVerify figure is too small to read as a real speed-up. The packed tier reproduces too (+8.1%, +16.8%, +32.1%), all positive and in the C figures' order. Both protocol drivers pass the same rejection-rate gate (section 2.6). The same qualitative result, small adaptor-layer overhead, appears in both implementations and under the independent Criterion harness (fig. 3.2).

3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap at the core boundary; the size results show where the real cost lies. Table 3.4 decomposes every LAS object into bytes at the target setting, explaining *which* component drives the size. Sizes are fixed by the encoding, so they are exact and carry no dispersion.

Two points follow. First, the response z is essentially the entire signature (99.3% of it at every parameter set) since the challenge travels only as its short digest, so z is the main target for reducing LAS signature size. Second, the adaptor

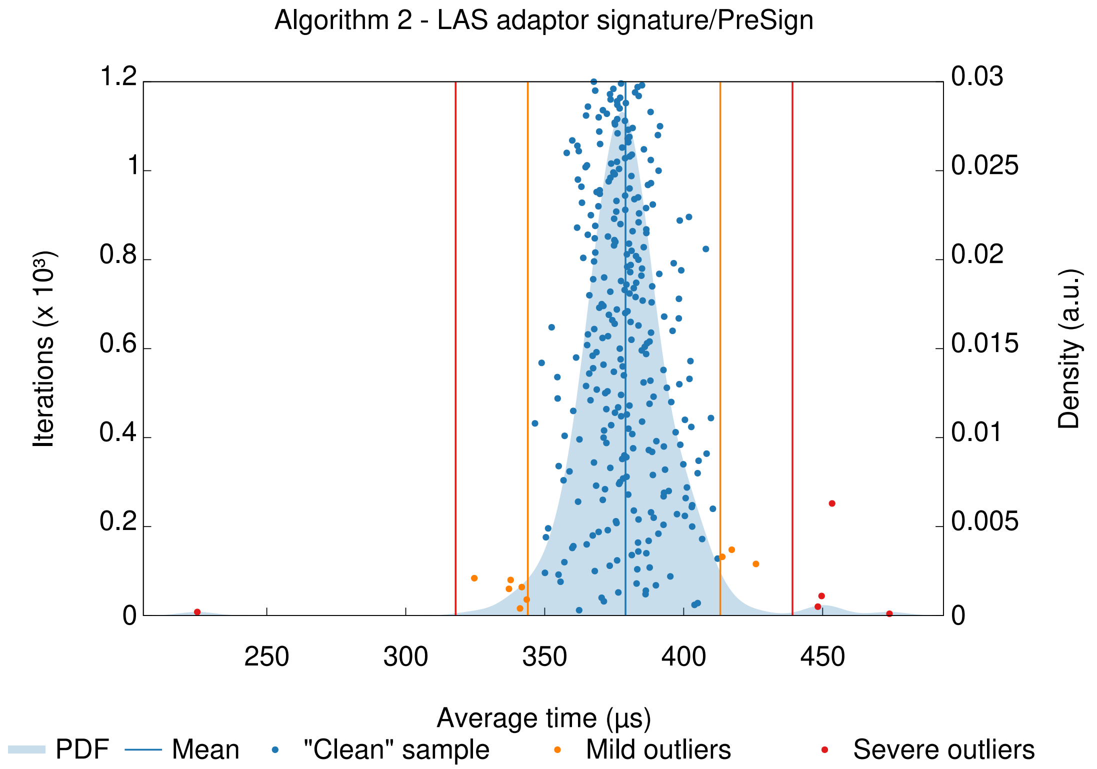

Figure 3.2: Criterion.rs sampling distribution for PreSign at the target setting: 300 timed samples (dots), the kernel-density estimate (shaded), the mean (vertical blue line), and Criterion's automatic outlier classification (mild / severe, orange / red). Four presentational changes were made to Criterion's SVG: the type enlarged, the legend repositioned beneath the plot, the margins re-flowed and cropped, and the exponent notation redrawn; plotted values, coordinates, scales and outlier classifications are unchanged. Of the 300 samples, 14 are classified as outliers (9 mild, 5 severe). This is the evidence behind the Criterion column of table 3.3.

layer additionally transmits a public-key-sized statement Y and a signature-sized pre-signature, the only objects a basic signature does not send; the swap also carries π.

The dominant cost of LAS is communication, specifically z and Y. At the core tier, the adaptor operations add at most +7.7% of compute, while the objects on the wire are kilobytes. That matters more in a blockchain setting than "the signature is 6736 bytes": bytes reaching a ledger are stored and replicated by every node, while the statement and pre-signature, though off-chain, still cost both parties bandwidth. Each response coefficient is bit-packed at a fixed width,

Table 3.4: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting, in packed wire bytes (not on-chain gas), with its notation, that of Esgin et al. [7], except that a signature carries the *digest* $\tilde{c}$, not the polynomial c (table A.2). "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. Three facts read off directly: the signature, pre-signature, and adapted signature are the same serialized size (adaptation changes the *value* of the response, not its length); the response z dominates every signature; and the only *public-key-shaped* object LAS adds over a basic signature is the statement Y (the protocol also transmits a signature-sized pre-signature, which the table marks as added). The sizes hold for the C and the Rust implementation alike: the wire encoding is byte-identical in both by construction, asserted programmatically on every Rust run (section 3.3). Sizes at every parameter set: table A.6 (appendix).

Object	Notation	Bytes	In the basic signature?
Public key	$pk = t$	4416	yes (shared)
Secret key	$sk = r$	704	yes (shared)
Challenge digest	$\tilde{c}$	48	yes
Response	$z \,/\, \hat{z}$	6688	yes
Signature	$(\tilde{c}, z)$	6736	yes (basic signature)
Statement	$Y = t'$	4416	LAS only (public)
Witness	$y = r'$	704	LAS only (private)
Pre-signature	$(\tilde{c}, \hat{z})$	6736	LAS only
Adapted signature	$(\tilde{c}, z)$	6736	LAS (verifies as basic sig)

with no entropy coding: the clearest opportunity for reducing that footprint (section 5.3).

The price of post-quantum: classical adaptor baseline. The second baseline is a classical secp256k1 ECDSA adaptor signature [14]: the natural functional counterpart of LAS, driving the same scriptless swaps but resting on discrete-log, not lattice, hardness. It measures the practical cost of moving from a classical adaptor signature to a post-quantum one. LAS is instantiated at *Simplified Dilithium-II* and only there: secp256k1 offers roughly 128-bit classical security, so those dimensions are the most like-for-like engineering match, while -III or -V would pit the classical scheme against a deliberately stronger configuration. The boundaries differ between the two libraries, so table 3.5 matches them per operation and the comparison is read conservatively: the conclusions rest on factor-level gaps, not percent-level differences.

Table 3.5: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) against LAS at the *Simplified Dilithium-II* parameters of table 2.1. Those dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim; the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings μs/op (mean ± SD, machine of record); sizes in bytes. The two libraries draw their measurement boundaries differently, so the final column is *tier-matched*: each LAS row taken at whichever of its two tiers corresponds to the classical library's boundary for that operation. The bold LAS entry in each row is the one that column uses. Why that matching is necessary, the boundaries themselves, the repetition counts, and the object-level differences behind the size rows are set out in appendix A.3. The two classical adaptor-layer ratios quoted in the text are each taken over *their own* base operation: PreSign over plain Sign (4.6×), PreVerify over plain Verify (4.0×). Both are *derived* from this library's single mixed native-API tier, not a paired, interleaved measurement of the kind section 3.2 reports for LAS. The corresponding LAS figures are the byte-tier ones, +19.7% for PreSign and +42.3% for PreVerify, that tier being the more conservative of the two reported for LAS.

	ECDSA adaptor (classical, hybrid native API)	LAS adaptor (post-quantum, core tier)	LAS adaptor (post-quantum, packed tier)	overhead LAS ÷ classical (tier-matched)
Computation ($\mu s/op$)				
KeyGen	13.0 ± 0.1	$\mathbf{29.7 \pm 0.7}$	103 ± 2	2.3×
Sign	17.9 ± 0.03	$\mathbf{274 \pm 4}$	441 ± 4	15×
Verify	26.1 ± 0.3	$\mathbf{61.9 \pm 0.2}$	123 ± 13	2.4×
PreSign	81.5 ± 1.3	284 ± 5	$\mathbf{526 \pm 4}$	6.5×
PreVerify	105 ± 3	64.1 ± 0.04	$\mathbf{260 \pm 8}$	2.5×
Adapt	1.4 ± 0.02	66.3 ± 0.03	$\mathbf{407 \pm 2}$	291×
Extract	14.3 ± 0.02	23.7 ± 0.02	$\mathbf{290 \pm 5}$	20×
Communication (bytes)				
Public key / statement	33	**2944**		89×
Secret key / witness	32	**512**		16×
Signature	64	**4640**		72×
Pre-signature	162	**4640**		29×

Two findings stand out. First, the post-quantum price is most pronounced in size: LAS's signature and public key are roughly 72× and 89× larger than the classical adaptor's, while every LAS operation stays well under a millisecond. Even the largest per-operation factor (Adapt, 291×) repays decomposition, because it compares two different *quantities of work*, not two speeds for the same work:

Algorithm 2 of [7] obliges ADAPT to run PREVERIFY before returning, so 64% of LAS's 407 μs is a full lattice verification and the rest is largely codec, whereas `secp256k1_ecdsa_adaptor_decrypt` deserialises the adaptor signature, performs one scalar inversion and multiplication, and conditionally normalises for low-S, leaving verification to a separate exported call [14]. Charging the classical side the verification LAS cannot skip puts the two at 106 μs against 407 μs — 4×, not 291×: a property of the two *interfaces*, not of lattice arithmetic. Second, the two schemes pay their adaptor overhead in opposite ways. The classical adaptor must attach a discrete-logarithm-equality proof [20], charging its own PreSign and PreVerify 4.6× and 4.0× their base operations, whereas LAS folds the statement into a hash it already computes. Relative to each scheme's own base operations, the lattice adaptor layer adds less computational overhead.

3.5 Testing the simplification: the adaptor with ML-DSA verification

Section 2.2 follows the simplified Dilithium-style base of [7], which omits Dilithium's hint vector, Power2Round and high/low-bit decomposition. In this simplified form, the exact identity $A\mathbf{z} - c\,t = w + Y$ is exposed. This section tests whether those omitted optimisations can coexist with the adaptor layer: the four adaptor operations were built around NIST ML-DSA *as FIPS 204 specifies it* [4], all three optimisations enabled. Three findings follow; the second pinpoints the boundary.

The naive port fails outright. Adding the statement to the challenge hash and changing nothing else (treating the hint machinery as a black box) yields pre-signatures that do not pre-verify at all (0/200), because the transmitted hint reconstructs HighBits(w) while the challenge committed to HighBits($w + Y$). Some modification is therefore unavoidable.

But the verifier is not what must change. Once the *whole* commitment path moves onto $w+Y$ (the committed high bits, the low-bits rejection test *and* MAKEHINT) and PRESIGN tightens its bound by the witness norm, the construction satisfies the full adaptor contract (13/13 items, including four tamper rejections and byte-identical determinism), and, decisively, the unmodified FIPS 204 verifier accepts the adapted signature (200/200). So PRESIGN and PREVERIFY require adaptor-specific algorithms, while VERIFY does not: a post-quantum adaptor swap could settle against a standard ML-DSA verifier. A matched no-statement baseline reproduces FIPS 204's own published repetition rates, which is what makes these rows attributable to the adaptor, not to a porting error.

The size gain is real but bounded, and it relocates the bottleneck. Measured against the scheme of record in one process, the adaptor layer stays single-digit on both constructions (+4.4% against +2.0% for PRESIGN per attempt), and per-signature cost is close to equal for opposite reasons: ML-DSA restarts about twice as often (5.2512 against 2.7348) but each attempt is roughly half the price. What ML-DSA buys is *bytes* (the signature falls to 3309 B from 6736 B), but the statement Y does not shrink at all (the same 4416 B), because Power2Round compresses a public key while Y enters the verification identity *before* any rounding. The signature-plus-statement payload $(\sigma + Y)$ therefore improves only to 0.69×, and Y becomes larger than the signature it accompanies. Compressing the signature moves the bottleneck to the statement without removing it, redirecting the size question of section 3.4 and the future work of section 5.3.

This is a functional demonstration, not a security argument: whether committing to HighBits$(w + Y)$ preserves ML-DSA's unforgeability is not analysed, and is exactly the kind of claim section 4.4 declares out of scope.

3.6 UTXO swap execution result

The protocol defined in fig. 2.4 was driven end-to-end on the model UTXO ledger of section 2.7. Two results matter. First, the protocol runs and every step is asserted, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement. Second, π is what makes Bob's final step *guaranteed*, not merely expected: by the Module-SIS uniqueness argument of [7] the extracted witness equals the proven ternary one, so the adapted signature is certain to clear the verification bound. Without π a malicious counterparty could claim and leave an unadaptable pre-signature behind. The proof measures 30711–30760 B on the wire at a knowledge error at most 2^{-127} and is exchanged off-chain only.

3.7 The swap on a UTXO chain: three configurations compared

The execution result above establishes that the protocol runs; this section measures what it *costs*, over the UTXO ledger and under the design of section 2.7: three configurations (table 2.2), of which only the $2 \to 3$ step is controlled. Timings are the mean ± sample standard deviation over 10 complete swaps at the *packed* tier, so they are not comparable with section 3.2. Both LAS configurations passed the rejection gate (2.8510 attempts per PRESIGN against the closed-form 2.77483).

The proof, not the signature, dominates execution time. Table 3.6 gives the per-phase timings behind this comparison. The role-A proof consumes 99.3% of end-to-end time in configuration 2 and 98.6% in configuration 3. Strip it out and the entire post-quantum signature workload (two key generations, a statement, two pre-signatures, a pre-verification, two adaptations, an extraction and two settlements) costs 4468.6 μs, about 5.5× the classical swap's 819.7 μs. That increase stays millisecond-scale: even at the packed tier the signature workload is not the bottleneck.

Table 3.6: Per-phase execution time of one complete swap, mean ± sample SD over 10 swaps (μs; AMD Ryzen 7 7745HX with Radeon Graphics). Unlike the Stage-1 tables, these timings are the *Rust* implementation's: the Stage-2 driver is a Rust program built on the port of section 2.5. Dashes mark phases a configuration does not have. End-to-end is context, not the headline.

Phase	Classical	LAS + Groth16	LAS + LaZer
KeyGen ×2 + Gen	58.4	540.7	503.2
Prove (π)	—	645621.6	216080.6
PreSign (u_1)	114.9	1065.3	1065.7
ProofVerify (π)	—	13718.1	90720.8
PreVerify (abort gate)	147.6	364.9	380.2
PreSign (u_2)	107.1	1070.1	966.7
Adapt (u_1)	138.4	392.5	380.5
Settle chain 2	51.9	253.0	239.9
Extract (u_2)	25.3	185.2	183.6
Adapt (u_2)	132.7	353.8	340.9
Settle chain 1	43.4	243.0	234.9
end-to-end	819.7	663808.3	311097.1
of which π	—	99.3%	98.6%
excluding π	819.7	4468.6	4295.7

The controlled comparison: LaZer proves faster, Groth16 proves smaller. Replacing Groth16 with LaZer (same signature, same $\mathbf{A}$, same inputs) makes the swap *faster* by 53.1% while making it *larger* by 61.9%: roughly three times as quick to produce a proof, yet emitting one about 240× bigger. Groth16 is *succinct* (three group elements whatever the circuit), but producing that proof means a multi-scalar multiplication over the whole constraint system, whereas LaZer's proof grows with the statement yet proves a lattice relation with lattice machinery, avoiding the encoding of a 23-bit modulus into a 254-bit pairing-friendly field. Verification runs the other way (13718.1 μs against 90720.8), succinctness paying off exactly where it is designed to.

Groth16 does not provide a post-quantum proof layer. Shor breaks Groth16's pairing-group discrete logarithms, so its proof is precisely the component a post-quantum adversary attacks: a post-quantum signature does not protect a swap whose fairness rests on a classical proof. Groth16 also needs a *trusted, per-circuit* setup (1.225142318s) whose secret must be destroyed for soundness, whereas LaZer's parameters are transparent.

Table 3.7: Communication cost of one complete swap, in bytes, observed from the transmitted buffers over 10 swaps. Off-chain messages are counted, not only what reaches a ledger. **A range indicates a message whose size varied between runs, and only configuration 3's proof does so**: LaZer packs the proof's Gaussian response vectors with a variable-length code and then reports the length its encoder actually reached, so the packed size depends on the values sampled for that particular proof. Every other object here has a size fixed by its encoding, and the two ranged totals inherit that single varying term.

Message	Classical	LAS + Groth16	LAS + LaZer
Off-chain (exchanged directly between the parties)			
statement Y	33	4416	4416
proof π	—	128	30711–30760
$\hat{\sigma}_1$	162	6736	6736
tx_1	114	4497	4497
$\hat{\sigma}_2$	162	6736	6736
tx_2	114	4497	4497
off-chain total	**585**	**27010**	**57593–57642**
On-chain (published to the ledgers)			
$tx_1 + \sigma_1$ (chain 1)	178	11233	11233
$tx_2 + \sigma_2$ (chain 2)	178	11233	11233
on-chain total	**356**	**22466**	**22466**
total per swap	**941**	**49476**	**80059–80108**

Communication is again the real price, with a usability consequence. Table 3.7 gives the corresponding byte counts. A classical swap moves 941 B; the fully post-quantum swap moves 80059–80108 B, about 85.1× more, and puts 63.1× more on the ledgers, the multiple that would matter economically on a Bitcoin-like chain, which charges by transaction size, not computation; the model ledger itself has no fee market. A fully post-quantum swap needs 311097.1 μs and tens of kilobytes of off-chain messaging per exchange, overwhelmingly the proof, against 819.7 μs classically: comfortable on a laptop, but a wide enough gap that a mobile wallet's assumptions would not port naively. No constrained device was measured, so that point should not be over-read.

3.8 Transaction structure and what a UTXO chain would charge

The sizes above are the model ledger's own encoding (appendix A.4). What a settled swap costs on a Bitcoin client is read from two *mined* transactions: an ordinary payment, and one leg of the swap of appendix A.11. Figure 3.3 compares them structurally and table 3.8 field by field. Both were mined on regtest, the swap leg on the node carrying the experimental rule.

The witness discount absorbs more than half of the post-quantum penalty. Before SegWit [21, 22] a signature travelled in the input script, where every byte was billed alike; the segregated witness bills witness bytes at one weight unit against four, and Taproot [23] supplies the script-path spend the swap input uses. Derived from the two mined transactions, the post-quantum settlement is 59.5× the classical one in raw bytes; in virtual size (what fees are charged on) only 26.4×, a post-quantum signature being *entirely* witness data. The discount removes 56 % of the size penalty; on the original structure the same settlement would pay the undiscounted rate throughout. It has no counterpart on the EVM, where calldata carries no comparable discount; section 3.9 measures what a gas-metered venue charges.

It fits the weight limits, not the element-size or verification rules. The mined settlement weighs 11,619 WU against the 400,000 WU standardness limit,

(a) Standard payment

`version`
input: outpoint, `sequence`
output: value, `scriptPubKey`
`locktime`
witness: σ, pk **107 B**

total 191 B = 110 vB

(b) Adaptor swap settlement

`version`
input: outpoint, `sequence`
output: value, `scriptPubKey`
`locktime`
witness: chunked σ and pk, tapscript, control block **11,289 B**

total 11,373 B = 2,905 vB

Never on chain: statement Y (4416 B), proof π, both pre-signatures $\hat{\sigma}_1, \hat{\sigma}_2$ — exchanged off-chain only. **Never transmitted at all:** the witness y, which u_2 recovers as y' from the published σ and its own $\hat{\sigma}_2$ via EXT.

Figure 3.3: Two *mined* transactions: an ordinary payment, and one leg of the swap settled in appendix A.11. **No field is added, removed or reordered**, and the base is byte-identical at 82 B: the swap condition needs no on-chain script of its own, which is what *scriptless* means. Exactly one row differs, and the difference does not come from the adaptor layer: the witness carries lattice objects, not an elliptic-curve signature and key, and because each stack element is capped at 520 B they arrive chunked, alongside the tapscript and its control block. Both transactions pay to the same output form, so the output does not grow; a settlement paying to taproot instead would add 12 B of base, which is not what was mined here. Sizes are the client's own, and the field layout follows [21, 22, 23]; table 3.8 gives every field.

2.9 % of it. The limit is a weight limit, not a gas limit, and one settlement transaction sits well inside it.

Two obstacles survive, neither caused by the adaptor layer. Bitcoin's deployed consensus rules verify no lattice signature: Script offers `OP_CHECKSIG` over ECDSA or BIP-340 Schnorr and nothing else, so configurations 2 and 3 could not settle on Bitcoin as it stands; the application setting in [7] assumes a UTXO chain "where the signature algorithm is replaced with a lattice-based signature scheme", and this study takes no position on which consensus route would deliver it. Separately, the objects collide with size limits: the signature is 13.0× over the 520 B consensus stack-item limit of [24] and 84× over the 80 B standardness limit [25]. Both travel

Table 3.8: Two mined transactions field by field, every figure as the client decoded it: an ordinary payment against one leg of the swap of appendix A.11. Total size, weight and virtual size are the client's own; the base is reconstructed from the fields and *cross-checked* against that weight, since BIP-141 fixes weight as three times base plus total [21]. "Witness stack" is the serialized items (for the swap leg the chunked signature and key, the tapscript and the control block), and "witness size" adds BIP-144's marker and flag [22]. Virtual size is the quantity Bitcoin charges fees on, and the gap between it and total size is the witness discount. **Scope**: both were mined on regtest, the swap leg on the node of appendix A.11 carrying the experimental rule.

Field	Classical (B)	LAS (B)
Base data — billed at 4 weight units per byte		
`version`	4	4
input count	1	1
input: outpoint + empty `scriptSig` + `sequence`	41	41
output count	1	1
output: value + `scriptPubKey`	31	31
`locktime`	4	4
base size	**82**	**82**
Witness data — billed at 1 weight unit per byte		
`marker` + `flag`	2	2
witness stack (2 items / 24 items)	107	11,289
witness size	**109**	**11,291**
total size	**191**	**11,373**
weight (WU)	437	11,619
virtual size (vB)	**110**	**2,905**

as length-checked 520 B chunks the new rule reassembles, satisfying the consensus limit without raising it. Both apply equally to an ordinary post-quantum payment: they are properties of lattice signature sizes.

Both obstacles were put to a real client. A stock Bitcoin Core node carries the lattice objects (refused by default relay policy, yet consensus-valid and mined) but verifies nothing: an ordinary Schnorr signature authorises it. Adding one consensus rule, an `OP_SUCCESS` opcode [24] over the byte interface of section 2.3, gives a node settling a two-leg swap with no elliptic-curve signature, 19 mutations tried being refused by it and accepted by an unmodified node, the control attributing the refusals to the new rule. The blocker is a consensus rule, not engineering. Its cost was measured: one verification runs 374 μs against the curve baseline, a BIP340 Schnorr check's 33.0 μs (11.3×, an overstatement at unmatched levels), and a signature that fails late costs about as much as one that verifies. The valid-path latency corresponds to a derived serial rate of 2.67k LAS verifications/s on this machine, not whole-node or network throughput. A patched node is not Bitcoin, so that conclusion stands; the exercise is functional, with no security argument (appendix A.11).

3.9 EVM verification as a separate venue check

As a separate check on deployability, native on-chain LAS verification was *implemented* and measured on a real Solidity stack: one claim transaction running full verification and releasing one leg's escrow costs ≈56.6 million gas (table 3.9;

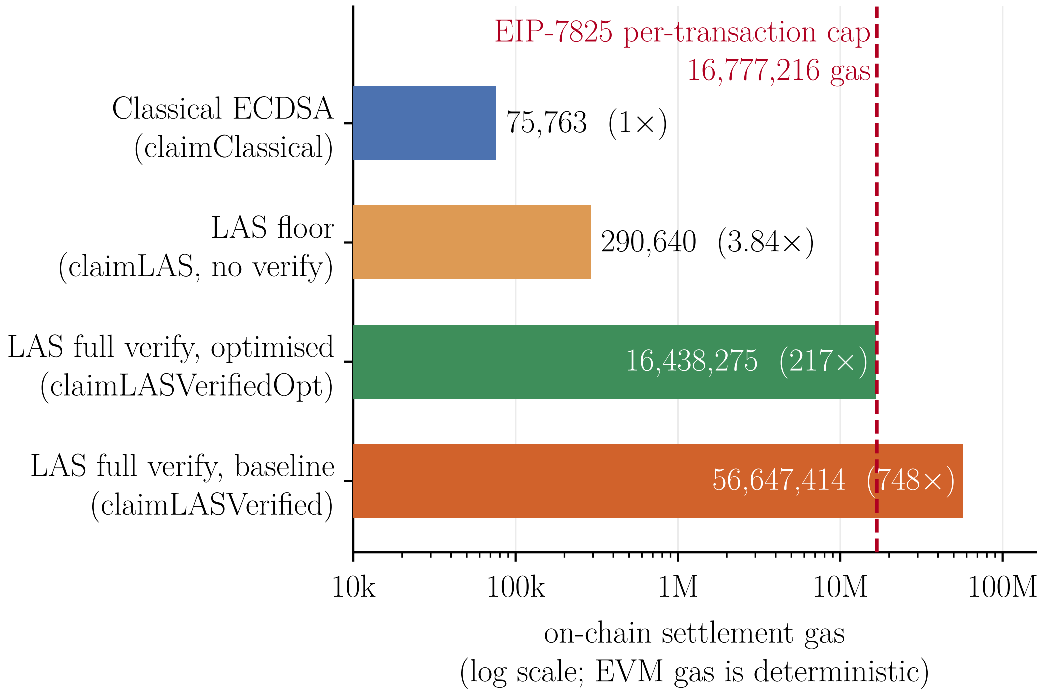

Figure 3.4: The claim paths of table 3.9 on a logarithmic axis, which is what makes the spread legible. The dashed line is the EIP-7825 cap on the transaction gas limit ($16\,777\,216 = 2^{24}$): the baseline verifier's bar crosses it, while the restructured path (same scheme, same predicate under the registration invariant of table 3.9, different expression) falls below. All four bars come from one `-gas-report` table and so are mutually comparable; the under-the-cap claim itself rests on the separate real-client run described in table 3.9.

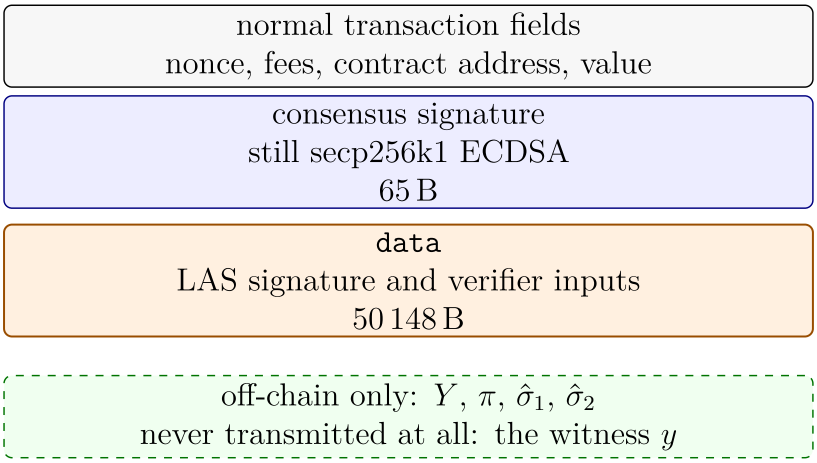

Figure 3.5: The same settlement on Ethereum: the claim transaction of the client run in table 3.9, read against the UTXO counterpart in fig. 3.3. Neither venue gains a field from the adaptor layer, but the lattice signature lands in a different structural position. On a Bitcoin-like UTXO chain it must occupy the signature slot consensus checks, so the chain needs a new consensus rule before it can verify LAS. On the EVM the authorising transaction signature remains secp256k1; the lattice signature rides in `data` as an argument to a contract. Contracts therefore make Ethereum the more flexible venue and Bitcoin the more restricted one here. The 50 148 B is measured from the client receipt and is the whole calldata payload, not the signature alone.

fig. 3.4), ≈748× the classical claim and over the EIP-7825 cap. The application is built on the UTXO setting assumed in [7] (section 2.7); this section measures what a gas-metered venue charges for the same verification.

That figure measures one *implementation*, not the scheme. A restructured verifier (same parameters, bounds and challenge preimage, differing in how the computation is expressed) was mined by a real client, running full LAS verification and releasing the escrow at 16 413 275 gas: 97.8% of the EIP-7825 cap, which bounds the transaction gas limit and so covers calldata as well as execution. Its equivalence is conditional on a registration invariant, and both are set out in table 3.9. On-chain verification therefore fits in one transaction at *Simplified Dilithium-III* for the 32-byte signed message and signature instance measured, with a thin margin. Verification alone at *Simplified Dilithium-II*, charged as one transaction, is 65% of the cap; at *Simplified Dilithium-V* one transaction is exceeded, derived rather than measured (appendix A.3). The contrast with the application's own venue remains: the UTXO constraint is a transaction *weight* limit one settlement sits well inside (section 3.8). The same distinction is structural, not only numerical: fig. 3.5 sets the EVM claim transaction against the UTXO counterpart in fig. 3.3.

Table 3.9: On-chain gas for the swap's claim step, measured on a local EVM (`forge -gas-report`); EVM gas is deterministic for fixed bytecode, revision, inputs and state. **These are the `-gas-report` charge for each claim path, not transaction totals and not client measurements**: they come from one table and are therefore comparable with *each other*, but they are not directly comparable with a receipt, which is measured by a real client at a different accounting boundary and additionally carries the 21,000 intrinsic charge and the transaction's EIP-7623 calldata cost. The under-the-cap claim in the text therefore rests on the client receipt of 16 413 275 gas, never on this table. The floor path performs *no* lattice verification and is a strict lower bound; ECDSA verifies in the native `ecrecover` precompile, whereas both LAS rows run in EVM bytecode. The two verifying rows decide the same predicate as base-signature verification (section 2.1), the second restructuring that computation without changing it, but **only under a registration invariant that neither row checks**, and where it fails a *different* predicate is decided; the equivalence testing, the invariant and the cost of breaking it are set out in appendix A.3. The client run used anvil at a pinned `osaka` revision with EIP-7825 enforced, over 50 148 B of calldata. **Scope**: *Simplified Dilithium-III*, a 32-byte signed message, that revision, and one signature instance; two stages of the verifier branch on coefficient values, so variation across instances is unquantified. The 363 941 gas of headroom is *measured*; that it corresponds to a few hundred further bytes of signed message is *derived* from it and the measured per-permutation hash cost.

Claim path	On-chain verification	Gas	× classical
Classical ECDSA adaptor	full, `ecrecover` precompile	75 763	1×
LAS floor, no verification	none (calldata + `keccak256`)	290 640	3.8×
LAS full verify	full base-signature verification, in Solidity	56 647 414	748×
LAS full verify, restructured	full base-signature verification, in Solidity	16 438 275	217×

3.10 Threats to validity

Several factors qualify these results. All timings are machine-dependent, mitigated by reporting standard deviations, fixing one machine and compiler, emphasising ratios over absolutes, and by the cross-implementation check (section 3.3). The parameter set's concrete security level is not formally established, security proofs being out of scope, so every cross-scheme claim is qualified by table 2.1; the modulus is FIPS 204's $q \approx 2^{23}$, not the 2^{24} of [7], which leaves correctness unaffected ($q > 2\gamma$) but changes the security margin. The extracted witness is exact here, whereas the relaxed setting of [7] allows witness noise that can grow across long payment paths.

Two further factors are specific to section 3.7. The UTXO ledger is a model rather than a node (appendix A.4): real transaction objects, real serialised sizes and the real leak channel atomicity depends on, but no fees or confirmation latency, so its byte counts are a model-level communication measure; section 3.8 separately reports Bitcoin's fee-relevant virtual size. That limit belongs to the venue, not the study: configurations 2 and 3 need a consensus change, built and run in appendix A.11. Finally, the Stage-2 timings come from a single desktop machine (AMD Ryzen 7 7745HX with Radeon Graphics), so performance on mobile or other constrained hardware was not measured.

Chapter 4

Evaluation

4.1 Evaluation strategy and its justification

The project set out to measure what adaptor functionality *costs*, so the evaluation was built around one requirement: every measured difference had to have a single attributable cause. Four decisions follow from it.

Correctness comes before cost. Performance figures for an incorrect implementation are worthless, and no formal proof was in scope, so correctness rests principally on differential evidence: a pinned known-answer value reproduced byte for byte by a second, independently written implementation. Internal self-consistency is not enough, and one error described below left every primitive passing in isolation while the assembled whole was wrong.

The primary comparison is controlled. LAS is measured against its own simplified base at identical algorithm, parameters and primitives, so the paired difference estimates the cost of the adaptor layer and of nothing else. Measuring against optimised CRYSTALS-Dilithium instead would confound that cost with parameter, packing and hint-vector differences; that contrast is therefore kept only as context, and the classical ECDSA adaptor only as a functionality-matched price of post-quantum.

Every measurement declares its boundary, because an undeclared boundary misleads quietly, as it did twice here before the tiers were fixed. For the same reason each run gates a directly counted rejection rate against its closed-form prediction rather than inferring acceptance from a timing ratio.

Metrics follow the venue. The application study reports execution time and communication bytes, counting off-chain messages, because the evaluated UTXO ledger carries no gas metric and because off-chain messaging is where the post-quantum cost lands.

4.2 Achievement of the objectives

Table 4.1: Each objective of section 1.3 against the evidence for it, which spans the introduction, chapter 3 and the appendix.

Objective	**Evidence**
1. Literature and scheme selection	Adaptor signature chosen for its direct relevance to atomic swaps; LAS [7] for the shortest path from a standardised lattice signature (section 1.2).
2. Additive implementation	Lattice core reused byte-for-byte with no upstream function modified, audited by a two-branch diff; adaptor layer, wire encoding and application are new code (table A.7).
3. Validation	Functional, serialization, tamper and known-answer tests pass (section 3.1); an independent Rust implementation reproduces the wire format and the pinned value byte-for-byte (section 3.3).
4. Fair benchmarking	Adaptor overhead isolated per operation at two declared tiers (table 3.2), against two further baselines, with the rejection rate gated against theory and the relative conclusions reproduced by the Rust port (table 3.3).
5. Atomic-swap demonstration	End-to-end two-party, two-chain swap settling both legs and asserting unspendability before adaptation (section 3.6), extended to a measured three-configuration study on a UTXO ledger (section 3.7).

Fairness was not simply asserted. The standalone evaluation holds algorithm, parameters and primitives fixed; in the application study the step from Groth16 to LaZer shares one public matrix and byte-identical inputs, verified at run time, while the classical configuration rests on a different curve and is a whole-stack contrast rather than a controlled one. The figures are then defended from two independent directions: a closed-form gate on the rejection rate, and re-measurement under a different compiler and an independent statistical harness. That second check is also where the claims stop. Absolute per-operation times do not reproduce across the two implementations and are not offered as a comparison between them; what reproduces, and what the conclusions rest on, is the relative cost of the adaptor layer.

The demonstration was built in the UTXO setting assumed in [7], then measured against a gas-metered venue for contrast. There a *complete*, validated native LAS verifier was charged ≈56.6 million gas for its claim path, beyond the EIP-7825 per-transaction cap, though that figure is a `-gas-report` charge rather than a transaction receipt. The cap turned out to bound the *implementation* rather than the scheme: a restructured verifier was mined by a real client inside it, at the target parameter set, for the message length and single signature instance measured, and only under the registration invariant that table 3.9 records as unchecked. The venue decision rests on cost, not on impossibility.

4.3 Implementation challenges

PreSign must reject at $\gamma - \kappa - 1$, one tighter than eq. (2.1). At the looser bound the adaptor operations can still succeed while the adapted signature later exceeds the norm ordinary Verify accepts; local adaptor tests did not expose it, and accounting for the witness norm identified the tighter bound. A related assumption, that Dilithium's hint and bit-splitting optimisations had to be disabled, also proved too strong: carrying the commitment path on $w + Y$ satisfies the tested adaptor contract with them enabled (section 3.5).

Implementing the scheme independently in Rust required byte-identical agreement, including deterministic sampling and serialization; the pinned known-answer value made that agreement directly testable (section 2.5). Benchmarking raised a separate problem, because rejection sampling makes sign-class timings right-skewed: acceptance inferred from a timing ratio was biased, and the two timed loops initially performed unequal work. Attempts are therefore counted directly, checked against theory, and measured at declared boundaries (section 2.6).

The Groth16 circuit initially cloned growing linear combinations while constructing dense rows, making the work quadratic in row width; building each row once through the lower-level constraint interface reduced it to linear cost. In Solidity, an already-NTT-domain public matrix was transformed again, producing a verifier that disagreed with the C reference; comparing the recomputed challenge digest against C exposed the error, the corrected port was validated in stages, and the verifier was then restructured to fit the per-transaction gas cap (section 3.9). Bitcoin imposed different constraints: Script has no native lattice-signature verifier, and its 520 B element limit prevents LAS signatures and public keys from being supplied as single elements. The integration therefore chunks these objects and reassembles them inside an experimental patched consensus rule (section 3.8).

4.4 Limitations

The threats of section 3.10 bound what the results *mean*, and three further limitations belong to the artefact itself, not to the measurement. The EVM verifiers reuse lattice primitives from a library its authors describe as not production-ready, so the gas figures characterise a numerically-correct verifier rather than a deployable one. The hint-free construction yields larger keys and signatures than optimised Dilithium: cryptographic optimisation traded for a transparent and testable artefact. Section 3.5 quantifies that price and shows it recoverable in principle, but only for the signature and not for the statement.

Configuration 2 gives up something different. Groth16's soundness rests on a pairing assumption Shor breaks, so a post-quantum signature does not protect a swap whose fairness rests on a classical proof. That configuration is the intermediate point which makes the controlled proof-system comparison possible, and is reported as that and nothing more.

Chapter 5

Conclusion, critical reflection, and future work

5.1 Conclusions

The aim was met within the prototype scope declared for it. The lattice core is reused unchanged; the adaptor layer, the wire encoding and the application are new and tested; functional, serialization, tamper and known-answer tests pass; a second implementation reproduces the pinned value byte for byte; and a two-party, two-chain swap runs end to end.

What adaptor functionality costs over a plain signature, and what a post-quantum operation costs over a classical one, are answered with measured data at declared boundaries. Over its own simplified base at identical parameters, the adaptor layer adds +1.8%, +7.4% and +7.7% for PreSign, PreVerify and Adapt at the core tier, staying within +7.7% across the parameter sweep and reaching +19.7–87.8% once the wire codec is counted, the increase there being codec work rather than arithmetic. Communication is where the cost actually falls: the signature is 99.3% response vector, with the statement Y on top of it. The classical comparison points the same way from the other side, its discrete-logarithm-equality proof making that adaptor layer proportionally the more expensive of the two.

The implementation follows the simplified Dilithium-style base of [7], and rebuilding the adaptor path on ML-DSA showed that the simplification is not functionally required for final verification. PreSign and PreVerify do need adaptor-specific algorithms, since the naive port fails outright, but Verify does not: a standard FIPS 204 verifier accepts the adapted signature. That route would halve the signature at close to equal computation and leave Y untouched: the communication bottleneck would move to the statement, not disappear.

Taking the protocol to the UTXO setting assumed in [7] sharpens the conclusion. At application scale the signature is not the dominant cost at all: the role-A proof consumes 99.3% of a Groth16 swap and 98.6% of a LaZer one. Changing only the proof system makes the fully post-quantum swap 53.1% faster end to end and 61.9% larger in communication; it needs no trusted setup and is the only one of the two Shor does not break, paying for that in bytes and in slower proof verification. The principal qualification is scope: no concrete security is formally analysed, and nothing settled on a live network.

5.2 Critical reflection

5.2.1 What was achieved

The decision that shaped everything else was additive construction on a standardised reference, with no lattice arithmetic reimplemented. It made the implementation tractable and its provenance auditable: the security-critical arithmetic is byte for byte the published CRYSTALS-Dilithium reference, and a two-branch diff shows exactly what was added. Being language-agnostic, that decision also made the second implementation cheap, which turned self-consistency into cross-implementation evidence and exposed the timing conclusions to an independent harness. Pairing a pinned known-answer value with an independent reimplementation proved a decisive way to validate a cryptographic port.

Two things went better than expected. Comparing LAS with its parameter-matched simplified base separated the adaptor overhead from the cost of Dilithium's omitted optimisations, which is the comparison every headline figure depends on. And counting rejection attempts rather than inferring them from timing corrected a materially wrong earlier figure; it is now a gate every run must pass.

Two negative results are worth as much. Compressing the statement inside this construction, and proving the relation under a succinct post-quantum system, were both implemented and neither paid off. Each now names the mechanism that defeats it, which closes the first direction outright and the second at the scale measured here.

5.2.2 What fell short

Three shortfalls remain beyond the scoped limitations of section 4.4.

On-chain settlement was measured, then solved narrowly, and still not deployed. The first Solidity verifier was charged far above the cap in `-gas-report` accounting rather than in a mined receipt, and the wrong lesson was drawn from it: the gap was treated as structural because SHAKE256 is not EVM-native. Restructuring the verifier closed it, at the target parameter set, for one signature instance and one message length, and only under a registration invariant no contract checks. What remains unsolved is cost: settlement is still far more expensive than a classical claim, and the margin is thin.

The applications remain exploratory, since Stage 2 runs over a ledger model and the real-client runs are local. For the UTXO study that is partly forced: Bitcoin Script cannot verify a lattice signature, so the post-quantum configurations need a consensus change to realise the setting assumed in [7]. These measurements establish implementation-level feasibility, but they are not deployment figures and not upper bounds.

The concrete security of the chosen parameter set is unanalysed by design. That sanctioned scope bounds what the numbers mean: they characterise a faithful, testable artefact, not a parameter set whose deployment security this study establishes.

5.2.3 What would be done differently

Application target selection should have come first. Too much effort went into Solidity before its cost was known, when the question that mattered was where a post-quantum swap actually spends. Starting from the UTXO setting assumed in [7] would have reached the three-configuration study sooner, leaving the EVM as the gas-metered venue check it eventually became.

Measurement boundaries should have been fixed before any benchmarking began: the biased acceptance estimate and the asymmetric timed loops both followed from not doing so. And the ML-DSA experiment should have preceded the written justification of the simplified base. Running it took days, and it overturned a belief the report had already argued at length.

5.3 Future work

- **A statement that is small by design.** With the signature halved and Y untouched, the statement becomes the dominant object. Compressing Y *inside* this construction was tested and fails at both relevant boundaries: ordinary verification reconstructs the commitment with the true Y, while EXTRACT checks the exact relation. Deriving Y from its seed instead reveals the witness. What is open is a different hard relation whose statement is small by construction.
- **Reduce the proof.** π must prove knowledge of a short witness to Y [7]; hiding it is required by the swap, while succinctness was this dissertation's added goal. That encoding loses to the deployed prover on measured prove and verify time, and on size by the library's own estimate rather than a packed proof: succinctness is asymptotic, and one role-A instance is far too small to reach it. What remains open is a proof small at *this* scale.
- **Reduce the cost of one-transaction verification.** Fitting under the cap was proposed here and then answered, though only for one signature instance and message length at both measured sets, verification alone at the smaller, with the largest exceeded by derivation. Fitting is not the same as being worth doing: settlement remains far above a classical claim, and the hash is the dominant measured term. A draft ML-DSA verification precompile covers NIST level II only [26] and was declined from the next scheduled upgrade [27]; it would not verify LAS, bearing instead on the ML-DSA route above, whose adapted signatures a stock FIPS 204 verifier accepts.
- **Analyse the security of the ML-DSA variant.** Section 3.5 establishes that committing to $\mathrm{HighBits}(w + Y)$ is *functionally* sound; whether it preserves ML-DSA's unforgeability argument is unexamined, and is the prerequisite before that route could be recommended over the simplified scheme.
- **Take the UTXO study to a live network.** *Signet* would supply the classical configuration with fees, propagation and confirmation latency absent from the model. Configurations 2 and 3 cannot yield deployment figures on a stock Bitcoin test network because the blocker is consensus, not engineering. Appendix A.11 adds that rule experimentally and settles a lattice-authorised two-leg swap that deployed Bitcoin cannot verify.

Bibliography

[1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999.

[2] R. Babbush, A. Zalcman, C. Gidney, M. Broughton, T. Khattar, H. Neven, T. Bergamaschi, J. Drake, and D. Boneh, “Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations,” *PRX Quantum*, vol. 7, Aug. 2026. Preprint: Cryptology ePrint Archive, Paper 2026/625, `https://eprint.iacr.org/2026/625`.

[3] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, “Transition to post-quantum cryptography standards.” NIST Internal Report 8547, initial public draft, Nov. 2024. `https://csrc.nist.gov/pubs/ir/8547/ipd`.

[4] National Institute of Standards and Technology (NIST), T. Dang, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, and A. Robinson, “Module-lattice-based digital signature standard,” Aug. 2024.

[5] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.

[6] M. Buser, R. Dowsley, M. Esgin, C. Gritti, S. Kasra Kermanshahi, V. Kuchta, J. Legrow, J. Liu, R. Phan, A. Sakzad, R. Steinfeld, and J. Yu, “A survey on exotic signatures for post-quantum blockchain: Challenges and research directions,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–32, 2023.

[7] M. F. Esgin, O. Ersoy, and Z. Erkin, "Post-quantum adaptor signatures and payment channel networks," in *Computer Security – ESORICS 2020*, pp. 378–397, Springer International Publishing, 2020.

[8] L. Valenta and K. Guthrie, "Post-quantum authentication to origins is now supported." Cloudflare Blog, July 2026. Published 29 July 2026. `https://blog.cloudflare.com/post-quantum-authentication-to-origins/`.

[9] A. Poelstra, "Scriptless scripts." Presentation, Milan Bitcoin meetup / Blockstream, 2017. `https://github.com/apoelstra/scriptless-scripts`.

[10] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized channels from limited blockchain scripts and adaptor signatures," in *Advances in Cryptology – ASIACRYPT 2021*, pp. 635–664, Springer International Publishing, 2021.

[11] V. Lyubashevsky, "Lattice signatures without trapdoors," in *Advances in Cryptology – EUROCRYPT 2012*, pp. 738–755, Springer Berlin Heidelberg, 2012.

[12] E. Tairi, P. Moreno-Sanchez, and M. Maffei, "Post-quantum adaptor signature for privacy-preserving off-chain payments," in *Financial Cryptography and Data Security*, pp. 131–150, Springer Berlin Heidelberg, 2021.

[13] R. Kysil, I. A. Seres, P. Kutas, and N. Kelecsényi, "poqeth: Efficient, post-quantum signature verification on Ethereum," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '25, pp. 327–343, ACM, 2025.

[14] Blockstream Research, "secp256k1-zkp: ECDSA adaptor signature module." Software library, 2026. `https://github.com/BlockstreamResearch/secp256k1-zkp`, commit 95b9835 (3 June 2026), vendored at `third_party/secp256k1-zkp`.

[15] CRYSTALS team, "CRYSTALS-Dilithium reference implementation." Software repository, 2023. `https://github.com/pq-crystals/dilithium`; the pristine upstream tree is vendored in this project, imported at repository commit 2374d22 and preserved on the `dilithium-baseline` branch.

[16] Lazer-crypto project, "LaZer: a library for lattice-based zero-knowledge proofs." Software library, 2025. `https://github.com/lazer-crypto/lazer`; accompanies the CCS 2024 paper "The LaZer Library: Lattice-Based Zero Knowledge and Succinct Proofs for Quantum-Safe Privacy".

[17] E. Schorn, "fips204: FIPS 204 ML-DSA in pure Rust." Software repository, 2025. `https://github.com/integritychain/fips204`, vendored at commit c948882.

[18] B. Heisler, J. Aparicio, and contributors, "Criterion.rs: statistics-driven micro-benchmarking for Rust." Software repository, 2026. `https://github.com/criterion-rs/criterion.rs`, version 0.8.2 (released 4 February 2026).

[19] S. Riou, J.-Y. Park, L. Anwar, A. Poschmann, and M. Hutter, "Apples, oranges, and signatures: Pitfalls and methodology in ML-DSA benchmarking." Cryptology ePrint Archive, Paper 2026/1333, 2026. `https://eprint.iacr.org/2026/1333`.

[20] L. Fournier, "One-time verifiably encrypted signatures a.k.a. adaptor signatures." Manuscript, 2019. `https://github.com/LLFourn/one-time-VES`.

[21] E. Lombrozo, J. Lau, and P. Wuille, "BIP 141: Segregated witness (consensus layer)." Bitcoin Improvement Proposal, 2015. `https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki`; defines the witness field, transaction weight and virtual size.

[22] E. Lombrozo and P. Wuille, "BIP 144: Segregated witness (peer services)." Bitcoin Improvement Proposal, 2016. `https://github.com/bitcoin/bips/blob/master/bip-0144.mediawiki`; defines the marker/flag and witness serialisation format.

[23] P. Wuille, J. Nick, and A. Towns, "BIP 341: Taproot: SegWit version 1 spending rules." Bitcoin Improvement Proposal, 2020. `https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki`.

[24] P. Wuille, J. Nick, and A. Towns, "BIP 342: Validation of taproot scripts." Bitcoin Improvement Proposal, 2020. `https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki`.

[25] Bitcoin Core developers, "Transaction relay and mining policy limits (`src/policy/policy.h`)." Source code, 2026. `https://github.com/bitcoin/bitcoin/blob/v31.1/src/policy/policy.h`; `MAX_STANDARD_TX_WEIGHT`, `MAX_STANDARD_P2WSH_STACK_ITEM_SIZE` and `MAX_STANDARD_TAPSCRIPT_STACK_ITEM_SIZE`, both 80 B. Pinned to v31.1, the release the node experiments run.

[26] R. Dubois and S. Masson, "EIP-8051: Precompile for ML-DSA signature verification." Ethereum Improvement Proposals, no. 8051, Oct. 2025. Draft. `https://eips.ethereum.org/EIPS/eip-8051`.

[27] T. Beiko, A. Stokes, A. Dietrichs, Nixo, and P. Jayanthi, "EIP-7773: Hardfork meta — Glamsterdam." Ethereum Improvement Proposals, no. 7773, 2024. Records which proposals are scheduled for, or declined from, the Glamsterdam upgrade. `https://eips.ethereum.org/EIPS/eip-7773`.

Appendix A

Supporting evidence and technical details

This appendix carries the supporting implementation, methodology and reproducibility material that would interrupt the main report if presented in full: the provenance of the benchmarks and tests, the derivations deferred from chapter 2, the representative PreSign, Adapt and Extract routines, supplementary benchmark data, and the real-client blockchain experiments.

A.1 Provenance of the benchmarks and tests

This section records how the reported benchmarks and tests were produced. The application experiments — the Stage-2 swap, the on-chain measurements and the real-client node runs — are described in the later sections of this appendix.

Provenance of the measurements. The standalone-signature figures come from two independently built suites, one per implementation, whose logs are parsed together to produce this report's figures, tables and inline numbers. The upstream Dilithium reference is vendored at import commit `2374d22`; the classical baseline library is the BlockstreamResearch `secp256k1-zkp` `ecdsa_adaptor` module at commit `95b9835`; the proof-of-knowledge library is LaZer at commit `2fa3dfb`; and the Rust port builds on a vendored copy of the `fips204` crate (commit `c948882`). Both external libraries were used unmodified and at those pinned commits, not at their current default branches. The C toolchain was the system compiler (`cc`, Ubuntu GCC 13.3.0) at `-O3`; the Rust toolchain was stable `rustc`, version 1.96.0 for the reported run, in release profile.

What was executed. The first-stage evaluation ran the parameter-matched base-versus-adaptor benchmark at each of the four parameter sets, then the functional, serialization and tamper, and known-answer tests at the target setting. The serialization test was built at the target dimensions, which are distinct from the LAS-2020/845 dimensions. The proof-of-knowledge and end-to-end swap tests ran once the proof library had been built; three superseded programs from an earlier type layout were excluded, no longer compiling against the current headers. Logs and parsed tables were captured before any figure was regenerated from them.

The Rust port. The port was exercised with the standard Cargo tooling: the test gate first, because the benchmarks refuse to time unverified code, then the Criterion.rs statistical harness, the protocol driver that mirrors the C one, and the packed-size report. The gate skipped the vendored crate's hint and Power2Round conversion tests, which the LAS port does not use, having its own serialization module. Its known-answer test asserted the same pinned value as the C implementation.

The ML-DSA adaptor experiment. The experiment of section 3.5 was run and recorded separately, because it measures a different construction: its logs are kept apart from those above and its numbers are never mixed with them. A single invocation covered the hint diagnostic, the adaptor contract and the head-to-head benchmark against the implementation of record, at ML-DSA-44, -65 and -87. The variants and the measurement design are set out in appendix A.3.

A.2 The timing-benchmark procedure

The pseudocode below is the procedure both protocol drivers implement, the C driver and its Rust counterpart being deliberately line-for-line mirrors, backing the methodology of section 2.6. Ordering matters: correctness is asserted *before* timing on the very objects about to be timed, so no failure path is ever measured, and the rejection gate runs over the *timed* calls themselves, so the gate certifies the same data the tables report. The same procedure runs once per parameter set.

Listing A.1: The timing-benchmark procedure (both tiers). $R = 5$ repetitions; $N = 1000$ (sign-class) or 1000 (verify-class) iterations; the rejection gate follows eq. (2.2).

```
Input: parameter set (n, l, kappa); fixed pp seed; fixed 33-byte
   message
 1 Setup: expand A from the fixed seed; KeyGen; relation Gen (Y, y)
 2 Gate 0 (correctness): run the full adaptor contract on the exact
   objects about to be timed (PreVerify accepts; basic Verify rejects
   the pre-signature; Adapt verifies; Extract returns y exactly);
   abort the run on any failure
 3 for each operation op in {KeyGen, Sign, Verify, PreSign, PreVerify,
   Adapt, Extract}: # core tier
 4 for r in 1..R:
 5 a0 <- attempt counter # sign-class only
 6 t0 <- monotonic clock
 7 repeat N times: run op (structures in / structures out;
            fresh masking randomness inside every sign-class call)
 8 run_r <- (clock - t0) / N
 9 att_r <- (attempt counter - a0) # sign-class only
10 report mean +/- sample SD over run_1..run_R
11 Gate 1 (run validity): for Sign and for PreSign, the mean attempts
   per call over ALL R*N timed calls must satisfy
       |mean - E| <= 5 * E*sqrt(1-1/E) / sqrt(R*N)
   where E = 1/p_acc is the closed-form expectation; abort on failure
12 repeat steps 3-11 at the packed boundary (packed bytes in /
   packed bytes out, decode + encode inside the call)
13 emit packed sizes (communication is fixed: measured once, no SD)
```

The Criterion.rs harness measures the same operations under its own protocol (3 s warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals) and applies the same Gate 1; it shares no timing code with the drivers, which is what makes its agreement an independent check, not a re-run.

A.3 Methodology details

Why the per-attempt acceptance probability is secret-independent. Each coefficient of the mask is uniform on the $2\gamma + 1$ values of $[-\gamma, \gamma]$, and the secret-dependent term satisfies $\|c\,r\|_\infty \leq \kappa$, because the challenge has exactly κ nonzero coefficients of ± 1 and the secret is ternary. Each response coefficient $z_i = y_i + (c\,r)_i$ is therefore uniform on a window of $2\gamma + 1$ consecutive integers whose centre is displaced from zero by at most κ. Sign's acceptance interval $[-(\gamma - \kappa), \gamma - \kappa]$ contains $2(\gamma - \kappa) + 1$ integers and lies wholly inside every such shifted window, because the displacement has magnitude at most κ. Each coefficient is thus accepted with probability exactly $\big(2(\gamma - \kappa) + 1\big)/(2\gamma + 1)$, independently of the secret — which is precisely why rejection sampling makes the published response simulatable, and hence why its distribution is independent of r. Raising this to the $(n + \ell)\,d$ coefficients of one attempt gives eq. (2.2). PreSign repeats the derivation at its own, tighter bound $\gamma - \kappa - 1$: that is why the two expectations differ slightly, and why the gate below is applied to each separately.

The run-validity gate in full. Over a run's timed sign-class calls the measured mean attempts $\bar{A}$ must satisfy $|\bar{A} - E| \leq 5\,\sigma/\sqrt{n_{\text{calls}}}$, where $E = 1/p_{\text{acc}}$ is the closed-form expectation of eq. (2.2) and $\sigma = E\sqrt{1 - 1/E}$ is the geometric standard deviation. The test is applied separately for Sign and for PreSign, at both timing tiers, in C and in Rust, and a run that fails it aborts instead of producing evidence. The resulting band is $\pm\!\sim\!6\%$ at the C drivers' 5000 timed calls and $\pm\!\sim\!1\%$ at the Criterion harness's $\approx\!10^5$, which is why only the latter resolves the 2% theoretical separation between Sign and PreSign. The gate is a consistency check on the observed rejection rate, not a proof of sampler correctness: it provides evidence that the rejection behaviour is consistent with theory and that rejection-sampling variation is sufficiently controlled for the reported mean.

The design of the ML-DSA compatibility experiment. Whether the optimisations the base of [7] omits can carry an adaptor layer (section 3.5) is settled by three variants of one implementation built on FIPS 204 [4] with the hint vector, Power2Round and high/low-bit split all enabled. A matched *control* removes the statement and is otherwise the same instrumented code, standing to the ML-DSA construction exactly as the base signature stands to LAS: same file, same helpers, only the statement differs. A *naive* variant changes one line, hashing the challenge over the shifted commitment while the committed high bits, the low-bits rejection test and MakeHint still use w. A *repaired* variant moves that entire path onto $w + Y$ and tightens PreSign's rejection bound by the witness norm. The stock FIPS 204 `crypto_sign_verify` is never modified: it is the acceptance oracle, which is what allows the decisive row to be read as a property of the construction itself; no re-implemented verifier is involved. The control is a precondition, not a result — were its signatures not accepted by that unmodified verifier, and did it not reproduce FIPS 204's published repetition rates, the mirror would be wrong and nothing else in the section interpretable. The diagnostic and the itemised adaptor contract each run over 200 randomised iterations per variant, at each of ML-DSA-44, -65 and -87. Cost is measured by the procedure of appendix A.2, both constructions in one process on one machine under one toolchain; within each construction the base and adaptor calls are paired and interleaved per repetition, so drift is common-mode there, whereas the two constructions themselves run in sequence, leaving drift between those two blocks a caveat the design does not cancel.

The Groth16 circuit. No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and it is far cheaper than the statement suggests. The map $r' \mapsto Ar'$ is linear with public coefficients, so each row is an affine combination of witness variables and needs no witness–witness multiplication — the expensive ingredient in a rank-one constraint system. The public linear map is still enforced, through linear constraints; the only non-linear and range constraints are the ternary bound, enforced as $r'^3 = r'$ per coefficient, and the reduction modulo q, enforced with a range-checked quotient per row. The constraint count is therefore linear in the parameter set's dimensions, not in their product, and the same statement would have been intractable had it involved products of two *secret* polynomials. Because the matrix is held in the number-theoretic-transform domain and is exported by neither implementation, it is recovered by evaluating the scheme's own linear map on unit vectors, so the circuit encodes the map the signature scheme actually computes with; no re-derivation is formed that could drift from it.

Measurement invariants of the UTXO study. Two invariants are enforced, not assumed. A phase must be timed in *every* run or in none, so that no mean is silently taken over the subset of swaps that reached it; and communication subtotals are computed within each run before being reduced across runs, since summing each message's minimum would describe a swap that never occurred. One-time costs — expanding A, building the elliptic-curve context, and Groth16's trusted setup — are performed before timing begins and reported separately, being amortised over every swap, not paid per swap. The rejection gate applies here too, but on a dedicated batch of 2000 pre-signature calls, not on the two per swap the protocol itself performs: over so few calls the five-sigma band is wider than the distance from the expected attempt rate to a loop that never rejects at all, so a gate on that sample would have been vacuous.

Matching the boundaries of the classical comparison. The two libraries of table 3.5 do not draw their measurement boundaries in the same place, so the comparison states both. The classical column is the library's *hybrid* native-API boundary: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call (packed-like), whereas KeyGen, Sign and Verify exchange in-memory structs with their wire codecs outside it (core-like). LAS is therefore shown at both its core and its packed tier, measured on the C implementation, and the tier-matched column pairs each operation with whichever LAS tier corresponds to the classical boundary for that operation — the core tier for KeyGen/Sign/Verify and the packed tier for the four adaptor operations. Timings are the mean $\pm$ sample standard deviation over 10 runs $\times$ 1000 iterations classically and 5 repetitions $\times$ 1000/1000 iterations for LAS, on the same machine. Comparing every row against a single LAS tier instead would count the wire codec on one side only, for four of the seven operations, which is why the matching is done per operation, not per table. Two object-level differences lie behind the size rows: LAS KeyGen also produces the statement and witness, which take the public-key and secret-key size, and the classical pre-signature is a syntactically distinct object — an ECDSA signature plus a discrete-logarithm-equality proof — whereas the LAS pre-signature shares the signature format exactly.

What the restructured on-chain verifier changes, and how it was checked. Subject to the registration invariant set out in the next paragraph, which neither verifier checks, the two verifying rows of table 3.9 decide the same predicate as base-signature verification: decode, norm bound, $w' = A\mathbf{z} - c\,t$, and the SHAKE256 challenge check. The first mirrors those steps directly from the vendored ZKNox primitives and is validated against the C reference. The second re-expresses them: the public key is registered in the number-theoretic-transform domain and ct folded before a single inverse transform, parameters are read from calldata without being decoded into memory, and the sponge avoids the per-byte absorb. It reproduces the first verifier's packed w' byte-for-byte on the C golden vectors, and agrees with it on accept and reject for the golden signature and for a tampered response, challenge and message.

The registration invariant, and what is *not* enforced. Neither verifier derives its public parameters: the matrix arrives already transformed in both, and the restructured one additionally takes the public key twice — transformed for the arithmetic, packed for the challenge preimage $\mathrm{pack}(t) \,\|\, \mathrm{pack}(w') \,\|\, M$, which is the only one of the three the hash ever sees. Equality with base-signature verification therefore holds exactly where three things are true together: the registered matrix is $\mathrm{NTT}(A')$ for the agreed A', the registered key is $\mathrm{NTT}(t)$ for some key t, and the packed bytes that are hashed are $\mathrm{pack}(t)$ for that *same* t. The escrow contract commits to all three at funding and re-derives that commitment at claim time, which stops a *claimer* substituting; it establishes nothing about their mutual consistency, and no on-chain step recomputes one from another. The obligation rests on whoever funds the escrow.

What follows from breaking it is, in general, only that a *different* predicate is decided — not that a weaker one is, and no analysis here covers the general case. Some registrations are weaker, though, and one suffices to show the condition is load-bearing, not pedantic: register both transformed objects as zero and every product vanishes, leaving $w' = \mathbf{z}_{\mathrm{top}}$ and an acceptance test that anyone can satisfy for any in-bound $\mathbf{z}$, with no key and no signature. The escrow would then pay out without the adapted signature ever reaching a chain — and that publication is exactly what leaks the witness and makes the swap atomic. Custody is unaffected, since the beneficiary is fixed at funding and the claim entrypoint pays that address whoever calls it; what a bad registration costs its own funder is the atomicity the escrow was bought for. Recomputing $\mathrm{NTT}(t)$ from the packed bytes once at funding, off the verification path, would discharge the two conjuncts about t; the one about A' would need the untransformed matrix, or the seed it expands from, supplied at registration as well. Neither is implemented, neither is costed here, and the measured figures are for the verification path as it stands.

Carrying the one-transaction result to the neighbouring parameter sets. The restructured verifier fixes its dimensions at build time, so a second copy was instantiated at *Simplified Dilithium-II*: duplicated, not parameterised, so that nothing could move the Dilithium-III figures already measured, and given its own golden vectors exported from the C implementation. What is charged there is *verification only* — the calldata goes to a harness entry point that runs the verifier and returns its verdict, so none of the escrow work in the claim path is included: no context re-derivation, no state transition, no event, no payout. Charged as one transaction under EIP-7623 it comes to 10 956 784 gas, 65% of the cap, 5 820 432 short of it, with the standard branch rather than the calldata floor binding. It is therefore a harness computation over a narrower boundary than the Dilithium-III client receipt of section 3.9, and the two are not interchangeable. The accepted verdict is asserted before the figure is reported — a rejecting run would have measured the early exit rather than verification — though the assertion follows the measurement and does not precede it.

Simplified Dilithium-V was not built, and the asymmetry between the two directions is why a derivation could still settle it. Deriving that a set *fits*, without measuring an instance of it, needs an upper bound over the whole computation, and two stages resist one: the challenge sampler's rejection loop and the response decoder both branch on coefficient values. Deriving that a set is *exceeded* needs only a lower bound, which tolerates dropping exactly those stages. The derivation therefore pushes the calldata the way that favours fitting, subtracts both data-dependent stages whole, counts every remaining stage at zero growth — each being a loop over the dimensions with no value-dependent branch — and adds one measured quantity: the difference between the two challenge-preimage absorptions, taken against a fixed-size memory arena so that the scratch allocated inside the timed call cancels and does not inflate the difference. That this measured difference clears the gap the other terms leave is what the result rests on (a growth gate measuring the two absorptions against a fixed-size arena, and a separate arithmetic step assembling the bound from that gate's output). What follows is that one transaction is exceeded at that set; no gas total for it follows, a lower bound not being a value, and neither does any claim about what further optimisation would achieve there.

The Dilithium-II and Dilithium-III figures are each one signature instance, for the same reason those two stages resist an upper bound, and variation across instances is unquantified at every set, so its magnitude remains unknown in both directions. The Dilithium-V statement is not an instance at all, but a bound assembled from Dilithium-III measurements and one measured absorption difference.

One deliberate exception to reproducibility. Every other input derives from the pinned seed, but Groth16's trusted setup and each of its proofs draw fresh operating-system entropy. Both are security requirements, not preferences: a reproducible setup would leave the toxic waste recoverable and void soundness, and reusing proof randomness across statements breaks zero-knowledge. The measurement consequence is stated openly — proof *size* is unaffected, since a Groth16 proof is three group elements regardless, but proving *time* includes the entropy draw and so carries a little extra run-to-run variance.

A.4 The modelled UTXO ledger

The ledger behind section 2.7 is a model, not a node: it has no peer-to-peer layer, no mempool policy, no fee market, no reorganisations and no script interpreter. What it does provide is what the study needs — real transaction objects with canonical serialisation, a signature hash computed over the transaction rather than over an abstract message, and the public observability on which the protocol's atomicity depends, since a settled transaction's signature can be read back off the ledger by anyone, which is exactly how the witness leaks. An output carries an amount plus a spending condition: a signature check under a public key, with an optional timelocked refund branch.

One boundary belongs with the atomicity claim. In the measured swap each coin is funded to an output that its own holder's key controls, and no conflicting spend of a funded outpoint is modelled. What the runs establish is therefore that the honest ordering settles both legs and that the witness leaks as the protocol requires — not that a party who has already claimed is unable to race to respend its own funded outpoint. Figure 1 in [7] does not specify a funding or locking step, so adversarial funding races lie outside both the figure and this experiment.

Deploying instead on a real Bitcoin node is not a matter of effort. Bitcoin Script cannot verify a lattice signature, so configurations 2 and 3 could not settle there without a consensus change — the setting in [7] instead assumes a UTXO-based chain *whose signature algorithm is replaced with a lattice-based signature scheme*, which is why that sentence is stated as an assumption rather than left implicit.

A.5 Wire encoding and the typed codec

The packed field widths behind section 2.3 are 23 bits per coefficient for the public key and statement, 2 bits for the ternary secret and witness, and 18 or 19 bits for the response z depending on the parameter set, since that field must hold $2(\gamma - \kappa)$ and γ grows with $\kappa\, d\, (n + \ell)$. Validation is deliberately *asymmetric*, and the asymmetry is worth stating precisely because it decides where a bad object is caught. Decoding rejects a public-key or statement coefficient $\geq q$ and the invalid ternary code, but a signature decodes *permissively*: $\tilde{c}$ is copied raw and z is read with the FIPS 204 $\mathsf{BitUnpack}$ of [4]. The response band $[0, 2(\gamma - \kappa)]$ does not fill the fixed-width field that carries it, so some byte strings decode to a z outside the valid band. That is intentional: there is no decode-time norm test to fail. The norm rejection belongs to Verify, which recomputes the challenge from the decoded response and compares digests. Encoding is the strict direction: it refuses a non-ternary secret or witness and a response exceeding $\gamma - \kappa$, so the serializer cannot emit an out-of-band response, even though one can be handed to the decoder. The byte-level entry point The byte-level verification interface decides the predicate over these bytes, and it — not the decoder — is what rejects all 6736 tampered signatures in section 3.1.

One encoding decision is worth recording. Following FIPS 204 [4], the wire format carries not the challenge polynomial c but only the digest $\tilde{c}$ from which it is derived, verification re-deriving $c = \mathsf{SampleInBall}(\tilde{c})$ after decoding. This is a computation-versus-storage trade: storing the expanded polynomial would save that re-derivation on every verification, while storing the digest keeps the signature smaller.

The digest *width* is an implementation choice rather than a requirement of the LAS construction [7], which specifies H only as a random oracle onto the challenge set C and fixes no encoding. It is therefore taken from FIPS 204, whose $\mathsf{SampleInBall}$ consumes a seed of $\lambda/4$ bytes, with $\lambda = 128, 192, 256$ for ML-DSA-44, -65 and -87 giving 32, 48 and 64 bytes. The width tracks the LAS parameter set and not the reference build mode, since the sweep compiles several LAS sets against a mode chosen only to satisfy the reference's parameter header: the ML-DSA-65-aligned target set takes 48 bytes, the ML-DSA-44- and ML-DSA-87-aligned LAS sets take 32 and 64, and the historical LAS-2020/845 reference set, which corresponds to no ML-DSA parameter set, takes 32 by this project's choice. Matching these reusable parameters aligns the primitives and the challenge-digest strength only — LAS retains its own ternary secret distribution, rejection bounds, exact hint-free relation and adaptor algorithms — so it is not a claim that the scheme is ML-DSA-65 or inherits its security category.

A.6 The atomic-swap demonstration and proof of knowledge

The protocol defined in fig. 2.4 follows Figure 1 in [7] step for step, realised over the model ledger of appendix A.4. Two parties swap coins across two ledgers with no trusted third party and no swap-specific on-chain script — ordinary output scripts remain, which is what *scriptless* means here. Alice, the witness holder, commits first: she draws a fresh statement/witness pair (Y, y), produces a zero-knowledge proof π that Y has a *ternary* preimage $(\exists\, r' : \mathbf{A}r' = Y \wedge \|r'\|_\infty \leq 1)$, pre-signs the transaction spending her own coin, and sends $\{Y, \pi, \hat{\sigma}_1\}$ in one off-chain message. Bob pre-signs his own transaction against the *same* Y only after both π and Alice's pre-signature verify — the protocol's abort gate; at that point neither pre-signature is spendable. Alice, who knows y, adapts and publishes her signature to claim Bob's coin; Bob, holding the matching pre-signature, extracts y from that published signature and uses it to adapt and publish his own. In the measured executions both legs settle and the witness is recovered as the protocol requires; the adversarial case is outside what this demonstration establishes (appendix A.4). The demonstration prints this narrative and asserts every step, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement.

The proof π reuses the LaZer lattice zero-knowledge library [16]; no proof system was written from scratch — the same reuse posture applied throughout. Because LaZer proves per-partition binary coefficients or exact Euclidean norms, the ternary statement is encoded by binary decomposition, $[\mathbf{A} \mid -\mathbf{A}]\,(r_+ \,\|\, r_-) = Y$ with both halves proven binary, which is equivalent to knowledge of a ternary preimage $r' = r_+ - r_-$. The generated parameter set reports a knowledge error at most 2^{-127} under Module-SIS and a proof size of $32\,046\,\mathrm{B}$ — the parameter set's own figure, larger than the proof bytes observed on the wire in table 3.7 and not to be conflated with them — with proving and verifying each completing well under a second. A dedicated test suite asserts completeness, rejection of every sampled single-byte tamper, rejection against a wrong statement, and that the prover refuses a non-ternary witness. The modelled ledger of appendix A.4 additionally tests a separate timeout/refund edge case that Figure 1 in [7] omits. In this one-chain test, a coin is funded to an output claimable by the counterparty and refundable by its funder after a timeout; the counterparty never responds, an early refund is rejected as a *timing* failure, not a cryptographic one, and after the timeout the funder recovers its coin.

A.7 Test types and parameter notation

This section collects the testing taxonomy (table A.1), the parameter notation (table A.2) and the measurement boundaries (table A.3) used throughout the evaluation.

Table A.1: The four test types, what each asserts, and what it would catch. The functional suite's statement-binding clause — that a pre-signature *fails* basic verify — is the tripwire that distinguishes an adaptor signature from a signature; the known-answer value is what the second implementation is checked against (section 2.5).

Test type	Scale and assertions	Catches
Functional	1000 iterations per Dilithium mode (2, 3, 5), fresh parameters, keys and message each time; asserts the full adaptor cycle — pre-signature pre-verifies, *fails* basic verify, adapts to a valid signature, and extraction returns the witness exactly — plus a sign/verify round trip and one-bit-forgery rejection	any break in the adaptor contract; statement binding
Serialization	256 random instances; exact round-trip for keys and every signature variant	encoder/decoder asymmetry
Tamper	every one of the 6736 bytes of a valid packed signature flipped in turn; all must break verification; malformed public-key and secret-key encodings rejected at decode	a verifier that accepts tampered bytes; a key decoder that trusts its input
Known-answer	all inputs fixed, deterministic API, packed bytes of the public key, secret key, ordinary signature, pre-signature and adapted signature folded into one SHAKE256 digest compared against a pinned 32-byte value	any unintended change to sampling, algebra or encoding, on any machine

Interpreting those tests across the parameter sets requires a common notation. Table A.2 therefore collects the symbols used throughout the construction and the value each takes at the four evaluated settings; the figures and tables in chapter 3 refer back to these definitions and do not restate them. The notation follows the LAS paper [7]: the ring degree is its d, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. Two entries are exceptions, the challenge digest $\tilde{c}$ and its width $|\tilde{c}|$: the LAS construction [7] specifies H only as a random oracle onto the challenge set C and fixes no encoding, so both the digest and its width are implementation choices, taken from FIPS 204 wherever a set aligns with an ML-DSA one (appendix A.5).

Table A.2: Security and scheme parameters: the symbol, its meaning, and the value(s) it takes across the parameter sets, *listed in the row order of* table 2.1 (LAS-2020/845 reference; Simplified Dilithium-II; -III; -V). The notation follows the LAS paper [7]: the ring degree is its d, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. The exceptions are the challenge digest $\tilde{c}$ and its width $|\tilde{c}|$: the LAS construction [7] specifies H only as a random oracle onto the challenge set C and fixes no encoding, so both the digest and its width are implementation choices, taken from FIPS 204 wherever a set aligns with an ML-DSA one (appendix A.5). The figure and table captions in chapter 3 refer back to these names and values.

Symbol	Meaning	Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V
n	module rank: rows of A, length of public key t	4, 4, 6, 8
ℓ	extra columns of A' (secret-vector extension)	4, 4, 5, 7
$M = n + \ell$	response dimension (number of polynomials in $z, \hat{z}, y$)	8, 8, 11, 15
κ	challenge Hamming weight (number of ± 1 in c; Dilithium τ)	60, 39, 49, 60
γ	rejection / masking bound, $\gamma = \kappa\, d\, (n + \ell)$	122 880, 79 872, 137 984, 230 400
d	ring degree ($X^d + 1$)	256 (all sets)
q	modulus from the reused NTT, $q \approx 2^{23}$	8 380 417 (all sets)
c	challenge polynomial, the output of H: $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Expanded from $\tilde{c}$ by the signing and verifying paths after decoding, never transmitted	element of R_q (all sets)
$\tilde{c}$	challenge *digest*: the SHAKE256 output this implementation derives the challenge from, following FIPS 204's pattern $c = \mathsf{SampleInBall}(\tilde{c})$. This, not c, is what a signature carries	width below (all sets)

continued on the next page

Table A.2 — continued from the previous page

Symbol	Meaning	Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V
$\lvert\tilde{c}\rvert$	challenge-digest width in bytes: FIPS 204's $\lambda/4$ at the three ML-DSA-aligned sets; a project choice at the LAS-2020/845 reference set, which that rule does not cover	32 (project choice), 32, 48, 64

Notation fixes the symbols and parameter values used to describe each operation; it does not fix what a timing includes. Table A.3 closes that gap with the three measurement boundaries used in this dissertation. Comparisons are only ever made within one of them, which is why the classical baseline is recorded as a hybrid rather than folded into a single tier.

Table A.3: The measurement boundaries. Comparisons are only ever made within one boundary. The classical column is not a uniform tier but a *hybrid*, because `secp256k1-zkp` exposes the pre-signature only in packed form: the four adaptor operations behave like the packed tier and the three base operations like the core tier.

Boundary	What crosses it	What it answers
Core tier	in-memory structures in, structures out	what the adaptor *algebra* costs, isolated from encoding
Packed tier	packed wire bytes in and out, including the decode of every input and encode of every output (section 2.3)	what a wire or on-chain consumer pays per call with nothing cached; one operation's byte boundary, *not* a whole-protocol cost
Hybrid API (classical)	the boundary `secp256k1-zkp` itself exposes: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call; KeyGen, Sign and Verify exchange parsed structs with the wire codecs outside it	the classical cost as the unmodified library actually charges it

A.8 Supporting benchmark evidence

The tables in chapter 3 carry the primary results; this section gives the supporting computation data produced by the evaluation of appendix A.1.

A.8.1 Supplementary computation and rejection-sampling evidence

Table 3.2 and fig. 3.1 are the primary computation result in the main text. The supporting exhibits here give the parameter-sensitivity and rejection-sampling evidence behind the same conclusion: fig. A.1 shows that the core-tier adaptor overhead remains within +7.7% across the full sweep, while table A.4 and fig. A.2 compare the measured rejection behaviour with the geometric model and the validity gate.

Table A.4: Target-setting rejection statistics: the geometric model of eq. (2.2) against every measured sample. "Mean" is attempts per call and "Accept." is per-attempt acceptance; every measured row passes the five-standard-error gate of section 2.6.

Operation	Source (calls)	Mean	Accept.
Sign	geometric model, eq. (2.2)	2.719	36.8%
	C driver, distribution sample (2000)	2.662	37.6%
	C driver, timed-run gate (5000)	2.741	36.5%
	Rust driver, timed-run gate (5000)	2.726	36.7%
	Rust Criterion, gate (188791)	2.735	36.6%
PreSign	geometric model, eq. (2.2)	2.775	36.0%
	C driver, distribution sample (2000)	2.817	35.5%
	C driver, timed-run gate (5000)	2.707	36.9%
	Rust driver, timed-run gate (5000)	2.728	36.7%
	Rust Criterion, gate (188791)	2.779	36.0%

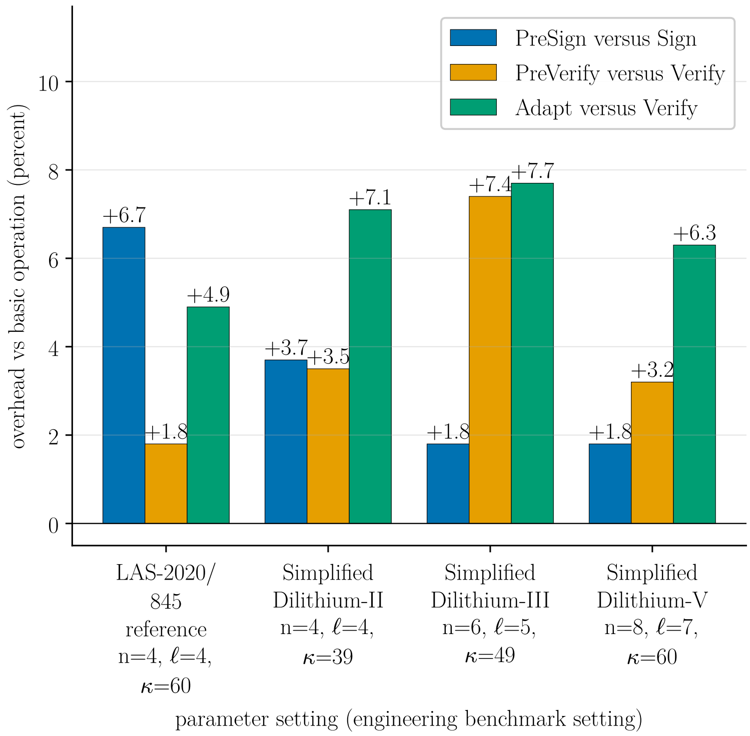

Figure A.1: Core-tier adaptor overhead across the four parameter sets of table 2.1. PreSign is compared with Sign; PreVerify and Adapt with Verify; Extract is omitted. Exact timings are in table A.5; target-setting overheads are in table 3.2.

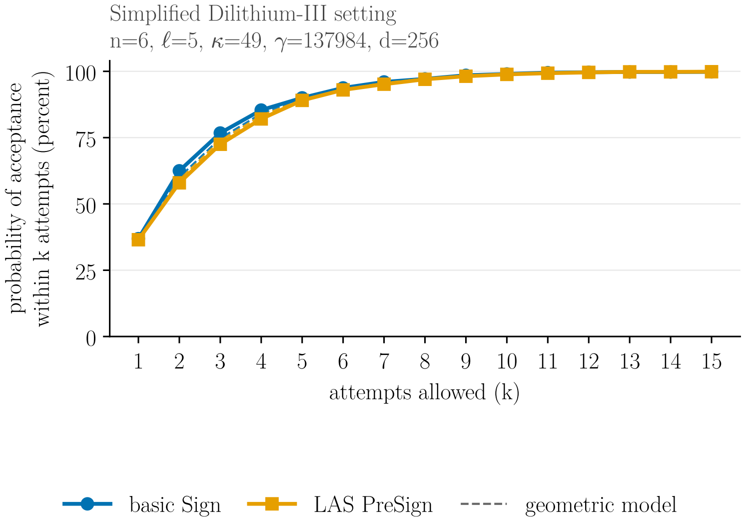

Figure A.2: Cumulative acceptance within k rejection-sampling attempts at the target setting. Solid curves are the C sample in table A.4; dashed curves are the eq. (2.2) model for Sign and the tighter PreSign bound.

A.8.2 Complete timing and component tables

Table A.5 lists every core-tier per-operation timing with its standard deviation (the data plotted in fig. 3.1, and the source of the core block of table 3.2); table A.6 gives the component byte sizes at the three Simplified Dilithium parameter sets (with the target setting reported in table 3.4).

Table A.5: Core-tier timings (μs per operation; mean $\pm$ sample standard deviation) from the parameter-matched benchmark. KeyGen, Sign and Verify are the reused basic operations; PreSign, PreVerify, Adapt and Extract are LAS adaptor operations. Setup is a one-time cost and is omitted.

Operation	LAS-2020/845 reference $(4, 4, 60)$	Simplified Dilithium-II $(4, 4, 39)$	Simplified Dilithium-III $(6, 5, 49)$	Simplified Dilithium-V $(8, 7, 60)$
Basic signature (reused unchanged by LAS)				
KeyGen	34.5 ± 3.4	29.7 ± 0.7	45.9 ± 1.9	70.6 ± 0.1
Sign	250 ± 6	274 ± 4	417 ± 11	508 ± 14
Verify	68.0 ± 1.8	61.9 ± 0.2	95.3 ± 0.5	138 ± 0.35
LAS adaptor operations				
PreSign	267 ± 7	284 ± 5	424 ± 6	517 ± 10
PreVerify	69.2 ± 0.3	64.1 ± 0.04	102 ± 2	142 ± 0.05
Adapt	71.3 ± 0.2	66.3 ± 0.03	103 ± 1	147 ± 0.17
Extract	26.5 ± 0.02	23.7 ± 0.02	37.2 ± 0.1	60.1 ± 0.03

Table A.6: Packed LAS component sizes. The challenge travels as the digest $\tilde{c}$, while z dominates each signature. The target-setting column corresponds to table 3.4.

Component	Simplified Dilithium-II $(4,4)$	Simplified Dilithium-III $(6,5)$	Simplified Dilithium-V $(8,7)$
$\tilde{c}$ (challenge digest)	32	48	64
z (response)	4608	6688	9120
Signature $(\tilde{c}, z)$	4640	6736	9184
z as % of signature	99.3%	99.3%	99.3%
Public key pk	2944	4416	5888
Secret key sk	512	704	960
Statement Y (LAS only)	2944	4416	5888
Witness y (LAS only)	512	704	960
Pre-signature (LAS only)	4640	6736	9184

A.9 Auditing the reuse claim

Table A.7 gives the per-file classification behind the "reused vs. modified vs. added" claim of section 2.2. The claim was established by keeping the pristine upstream tree on its own branch and diffing it against the project branch: the difference below print nothing for the lattice core, which is what makes "zero upstream functions modified" an auditable statement, not an assertion.

Table A.7: Components of the implementation, classified by how the reference is used. The lattice core is reused unchanged; inside the reference tree the only modified files are the build description, which gains targets, and the list of built artefacts to ignore. The branch diff also shows one deletion: an annotated duplicate of the reference signing module, carried in the pinned baseline, differing from it by comment blocks alone, named by no build rule, and removed here. The adaptor scheme, serialization, application integration, and evaluation harnesses are project-specific additions.

Category	Component	Role
Reused unchanged	polynomial and vector arithmetic, the number theoretic transform, modular reduction, rounding, packing, signing, hashing, parameters	the entire lattice core
Modified	the build description, the list of ignored build artefacts	new targets; no source
Added	the adaptor layer and its wire encoding, the relation and its zero-knowledge proof (over the external prover), the base signature, setup, tests and benchmarks	this work

The check is a difference between the two branches over the reference tree, taken twice: first by name and status, which enumerates everything that changed at all, and then over the eight modules making up the lattice core — polynomial and vector arithmetic, the number theoretic transform, signing, packing, modular reduction, rounding, and the parameter header. Empty output on the second difference is what establishes that the core is byte-for-byte the pinned baseline.

A.10 Selected code listings: PreSign, Adapt and Extract

Three routines carry the adaptor mechanism. PreSign folds the statement into the commitment before hashing and rejects at the tighter bound; Adapt adds the witness to the pre-signature's response; Extract recovers the witness by subtraction and confirms it against the statement. They compose the same reference-derived polynomial arithmetic used throughout the base path, with the LAS-specific response-vector subtraction required by Extract. The latter recomputes $A\,s$ using the same matrix and vector operations that key generation uses to form t.

Listing A.2: Selected implementation cores from the C adaptor module. The PreSign excerpt retains the adaptor-specific statement binding, challenge construction, response computation and rejection gate, while eliding the reference-derived matrix/vector setup around them; Adapt and Extract are shown in full, apart from abridged provenance comments.

```
/* --- PreSign: the adaptor-specific core, Algorithm 2 steps 2-7 (
    abridged) --- */
rej:
  ++las_attempts;
  las_polyvecm_uniform_Sgamma(y, mask_seed, mask_nonce++); /* y <-
      S_gamma^(n+l) */

  /* ... w = A y, via the reference-derived matrix and vector helpers
      ... */
  las_polyvecn_caddq(w);
```

```c
  /* THE core adaptor mechanism: the statement is folded into the
      commitment
   * before hashing. Sign hashes w; PreSign hashes w + Y, so the adapted
   * signature satisfies the ORDINARY Verify equation for the same
      challenge. */
  las_polyvecn_add(w_plus_t_prime, w, Y->t_prime);
  las_polyvecn_reduce(w_plus_t_prime);
  las_polyvecn_caddq(w_plus_t_prime);
  las_polyvecn_pack_w(w_packed, w_plus_t_prime);

  shake256_init(&state);
  shake256_absorb(&state, t_packed, sizeof t_packed);
  shake256_absorb(&state, w_packed, sizeof w_packed); /* the shifted w +
      Y */
  shake256_absorb(&state, m, mlen);
  shake256_finalize(&state);
  shake256_squeeze(c_tilde, LAS_CTILDEBYTES, &state);
  las_poly_challenge(&c, c_tilde); /* SampleInBall(c~) */
  c_hat = c;
  poly_ntt(&c_hat);

  las_polyvecm_pointwise_poly_montgomery(presig->z_hat, &c_hat, r_hat);
  las_polyvecm_invntt_tomont(presig->z_hat);
  las_polyvecm_add(presig->z_hat, presig->z_hat, y); /* z_hat = y + c r
      */
  las_polyvecm_reduce(presig->z_hat);
  if(las_polyvecm_chknorm(presig->z_hat, bound)) /* caller-supplied */
    goto rej; /* PreSign bound */
  memcpy(presig->c_tilde, c_tilde, LAS_CTILDEBYTES); /* sigma_hat = (c~,
      z_hat) */

/* --- Adapt --- */
int las_adapt(signature *sig, const pre_signature *presig,
              const uint8_t *m, size_t mlen, const statement *Y,
              const witness *r_prime, const public_key *pk,
              const public_params *pp) {
  if(las_preverify(presig, m, mlen, Y, pk, pp))
    return -1;
```

```
  memcpy(sig->c_tilde, presig->c_tilde, LAS_CTILDEBYTES); /* challenge
     preserved */
  las_polyvecm_add(sig->z, presig->z_hat, r_prime->value); /* z = z_hat
      + r' */
  las_polyvecm_reduce(sig->z);
  return 0;
}

/* --- Extract --- */
int las_ext(witness *s, const signature *sig, const pre_signature *
    presig,
          const statement *Y, const public_params *pp) {
  unsigned int i, j;
  poly s_1_hat[ELL], a_s[LAS_N];
  las_polyvecm_sub(s->value, sig->z, presig->z_hat); /* s = z - z_hat */

  las_polyvecm_reduce(s->value);
  for(j = 0; j < ELL; ++j) /* a_s = A s, formed */
    s_1_hat[j] = s->value[LAS_N + j]; /* as KeyGen forms t */
  las_polyvecl_ntt(s_1_hat);
  las_polyvec_matrix_pointwise_montgomery(a_s, pp->a_prime, s_1_hat);
  las_polyvecn_reduce(a_s);
  las_polyvecn_invntt_tomont(a_s);
  las_polyvecn_add(a_s, a_s, s->value);
  las_polyvecn_reduce(a_s);
  las_polyvecn_caddq(a_s);
  for(i = 0; i < LAS_N; ++i) /* accept iff A s == Y */
    for(j = 0; j < LAS_D; ++j)
      if(a_s[i].coeffs[j] != Y->t_prime[i].coeffs[j])
        return -1;
  return 0;
}
```

A.11 Settling on a real node

The Bitcoin transaction structures of section 3.8 follow the serialisation of [21, 22, 23]. This appendix records what happened when transactions of those shapes were put to a real Bitcoin client, in two stages that must not be conflated: the first carries lattice-sized objects past a *stock* node, the second verifies a lattice signature on a *patched* one. Full method, scope and caveats are in a separate write-up, and each stage was captured as its own evidence run.

Carriage on stock Bitcoin Core. Against an unmodified Bitcoin Core v31.1 regtest node (source commit `9be056a8a72b`), three transactions were constructed with the client's own test framework, offered to two nodes differing only in relay policy, broadcast and mined.

transaction	vsize (vB)	weight (WU)	default relay policy
1-in/1-out P2WPKH, signed by the node	110	437	accepted
1-in/1-out P2TR key path	111	444	accepted
carriage of a LAS signature and key	2,939	11,753	`bad-witness-nonstandard`

Three observations. The two reference spends reproduced the published figures the size model self-checks against, so that model is now one a client has confirmed, not one this study asserts. The carriage transaction is refused by default relay policy — the reason string is the node's own — yet is consensus-valid, was mined, and the default-policy node accepted the containing block: standardness and validity are distinct questions, and only the second bears on whether a post-quantum settlement is possible. Finally, because a signature of 6,736 bytes and a key of 4,416 bytes each exceed the 520-byte consensus limit on a witness stack item, they travel as 13 and 9 chunks respectively, which is why this transaction carries 25 witness items where an ordinary payment carries two.

Nothing here verifies a lattice signature. Stock Script has no such opcode; the spend is authorised by an ordinary Schnorr signature and the lattice bytes are discarded from the stack.

Verification on a patched node. [24] reserves the `OP_SUCCESS` opcodes as an upgrade path. Taking one of them (0xbb) and defining it as a lattice verification opcode over the byte interface of section 2.3, with the signed message being the transaction digest of [23], gives a node that can settle a spend carrying no elliptic-curve signature at all. Such a spend was accepted and mined (2,917 vB, 11,667 WU, 24 witness items); the consensus diff is recorded as a patch against the same release, and applies to a pristine checkout of it.

That a node accepts a transaction proves little on its own, so each case was put to both the patched node and a stock one. On the stock node the opcode is still `OP_SUCCESS`, so it accepts every variant unconditionally — which is what makes it a control, since it cannot be reacting to the signature. 9 mutations were tried: altering the payment's value or destination, spending a different coin, computing the digest against a false input value, presenting a signature made for another transaction or over something that is not a transaction digest at all, truncating or transposing a chunk, and substituting a key the output never committed to. Every one was refused by the patched node and accepted by the stock node, which attributes the refusals to the new rule, not some other defect.

A whole swap, not a single spend. Verifying one signature is not settling a swap. The property the protocol exists for is that the party who does *not* hold the witness — here u_2, since u_1 generates (Y, y) and holds y throughout — obtains it only from what the counterparty published. That was put to the patched client too, across two regtest chains sharing no coins and diverging from their first block. Each leg's signed message is its own transaction digest [23], so it commits to that leg's input, value and recipient; the two digests differ, which is what stops one signature settling both. PreSign and PreVerify run on both legs, u_1 adapts leg B and settles it, and the adapted signature is then read back out of the *mined* witness — its boundary located by the key hash the funding output committed to, not by an assumed signature length. Leg A is adapted with the recovered witness and settles on the other chain. Each leg is 2,905 vB, 11,619 WU and 24 witness items: 82 bytes of non-witness transaction, the two bytes BIP-141 adds as a marker and flag [21], and 11,289 bytes of witness. That witness holds the signature and the public key split across stack elements of at most 520 bytes, followed by the tapscript and its control block.

Two separate facts carry the provenance, and they should not be merged. The recovered signature is 6,736 bytes and byte-identical to what Adapt produced, which establishes that the chain carried exactly that signature and that the recovery read it faithfully. What attributes leg A's *witness* to the ledger is different: Extract runs as a separate program, given only the recovered bytes as an argument and unable to reach the copy the adapting party held. 19 negative controls were refused by the patched node and accepted by the stock one, among them the cross-leg case — leg B's settlement signature presented on leg A. Both were captured as a single evidence run.

One asymmetry with the Ethereum settlement of table 3.9 is worth stating, since it is a property of the venue, not of the scheme. Bitcoin binds the transaction, not the chain: the digest of [23] carries no chain identifier, whereas the Ethereum escrow's message is derived from `block.chainid` among other state. Offering leg B's settled transaction to the other chain is therefore refused because that coin does not exist there — which evidences two separate ledgers, and not a signature bound to a chain.

One swap across two kinds of ledger. Both legs above settle on Bitcoin. A further experiment put one leg on the patched client and the other on Ethereum, so that a single LAS instance — one public matrix and one adaptor statement, with one signing key pair per leg — is verified by two mechanisms that share no code: the consensus opcode above, and the deployed escrow contract of section 3.9, which receives the matrix as calldata. The Bitcoin leg settles first; its adapted signature is recovered from the *mined* witness, EXTRACT runs on those bytes as a separate program, and the recovered witness adapts the Ethereum leg, which settles inside one transaction under the EIP-7825 cap. What this adds to the two-leg run above is not the choreography, which is the same, but that the two halves are different ledger models: the venue changes while the cryptographic object does not.

The timeout side was exercised on both venues, each on a recovery-test object of that venue's own kind — a control coin on Bitcoin, a second escrow on Ethereum — not by refunding either settled leg, since both were claimed. Each was refused before its own deadline and, once matured, returned the coin. The two deadlines are set as absolute UNIX-second values so their ordering is checkable, with the leg settled first carrying the shorter one, following the classical setup recalled in Section 4.1 of [7] — the LAS choreography of Figure 1 restates no timeout, so this is an addition around that figure, not part of it. The ordering is *configured and checked, not enforced*: Bitcoin judges a timestamp locktime against median time past and the contract against `block.timestamp`, so the two are one numeric domain and not one clock, and neither venue inspects the other's deadline. None of this establishes a fair atomic swap. The Bitcoin claim leaf is a single-key check under the funder's own key, so the funder need not wait for the timeout, and the mempool exposes the adapted signature before confirmation, leaving a conflicting-spend race that a timeout does not close. Evidence: its own evidence run.

Three limits travel with this. A patched node is not Bitcoin, so the claim that configurations 2 and 3 cannot settle on Bitcoin as deployed is unaffected. Implementing one of the routes sketched in section 3.7 demonstrates feasibility and takes no position on which route should be adopted. And the exercise is functional: no security argument is offered for the new rule.

What the rule charges a validating node. Changing a consensus rule buys something and charges something, and the charge falls on every node that ever validates the chain, so the cost was measured, not left as a remark. The quantity that matters is script-validation time per input, since that is what scales with the number of signatures in a block. Timed over 10 repetitions $\times$ 200 iterations, paired and interleaved, one `OP_CHECKLASSIGVERIFY` costs $374{\pm}9\,\mu$s against $33.0{\pm}3.3\,\mu$s for a BIP340 Schnorr check and $31.7\,\mu$s for ECDSA — $11.3\times$ and $11.8\times$ respectively. Both sides start from *serialized* bytes and parse inside the timed call, as the interpreter does per input; timing a packed lattice verifier against an already-parsed curve key would charge the wire codec to one side only. The reject path is a *valid* signature checked against a different 32-byte digest, so every stage before the final challenge comparison must still run — a corrupted signature can trip an early structural check, which would measure how quickly the verifier gives up, not what a rejection costs. Rejection comes to $372\,\mu$s, $1.00\times$ an acceptance. The narrow reading is the only one supported: a LAS signature that fails at the final comparison costs a node approximately as much as one that verifies, so there is no early exit to be had on this path. It says nothing about the rule's denial-of-service surface more broadly — malformed inputs take other paths, which were not timed. The benchmark was captured as its own evidence run.

Three caveats bound those figures, and they are separate claims. First, the curve baseline is a pinned `libsecp256k1-zkp` — a fork of `libsecp256k1` running the same verification algorithms, but not the library the patched client links, so these are not Bitcoin Core's own numbers. Second, the security of the consensus modification remains unanalysed: a timing figure is not a safety argument, and soundness, denial-of-service surface and upgrade path are all untouched. Third, the two schemes are not at a matched security level — `secp256k1` offers roughly 128-bit *classical* security, whose engineering match is Simplified Dilithium-II, while the node compiles the rule at Simplified Dilithium-III because that is what it actually runs. The ratios therefore measure the curve against a deliberately stronger lattice setting and so *overstate* what the rule costs relative to a level-matched pairing; stating the direction of that bias is the point, and neither pairing is a security-equivalence claim. Finally, this is a per-input cryptographic cost only: block download, UTXO lookup, script parsing and signature caching are excluded, and the unmodified client pays those too.